

MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

School of Electrical Engineering

Program: ECM

Laboratory Record Book

Course Name: Digital Signal Processing

Course Code: ECM_3141

Semester: B. Tech: V

Academic Year: 2026-27



MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

School of Electrical Engineering

Vision

Excellence in electrical engineering education through continuous learning, research, innovation and teamwork.

Mission

To nurture qualified, motivated and competent graduates who demonstrate leadership, possess strong foundational knowledge with advanced competencies in electronics, communication technologies, instrumentation, VLSI technology, and Robotics & AI.

To advance research and innovation in electronics and allied fields aligned with emerging technologies, while upholding ethical standards, contributing to industry advancement, supporting the scientific community and benefiting society.



MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

Certificate

This is to certify that **Mr./Ms.** _____, **Registration No.:** _____, **Section:** _____, **Program:** _____, has satisfactorily completed all the experiments prescribed for the course _____ as part of the **B.Tech. Degree Programme** of the **Manipal Academy of Higher Education (MAHE)**. The laboratory work was carried out at the **Manipal Institute of Technology, Bengaluru, during the Academic Year 2026–27, in partial fulfilment** of the requirements for the award of the **B.Tech. Degree**.

MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

Contents

Expt. No.	Title	Page No.
	Course Outcomes and Course Description	i
	PO, PSO and Course Articulation Matrix	ii
	Lesson Plan	iii
	Justification Table for Course Articulation Matrix	iv
	Evaluation Plan & Rubrics	v
	Pre-Lab Preparation	vi
	Lab Observation	vi
	Lab Record	vi
	Safety Precautions	vii
	Lab Rules (Do's & Don't)	vii
	CO Wise Marks Distribution	ix
	Introduction to signal processing using MATLAB	1
1	Analysis of signals in time domain	9
2	Analysis of signals and systems in z-domain	17
3	Analysis of signals and systems in frequency domain	27
4	IIR Filter Design	39
5	FIR Filter Design	49
6	Image processing	59
	Introduction to Code Composer Studio (CCS)	69
7	DFT computation using TMS320C6748 DSP starter kit	71
8	IIR Implementation using TMS320C6748 DSP starter kit	75
9	FIR Implementation using TMS320C6748 DSP starter kit	79
10	Audio/Speech processing using TMS320C6748 DSP starter kit	83

Course Syllabus

Digital Signal Processing Lab

ECM_3141 **[0 0 3 1]**

Total Contact Periods: 20 Hrs

COURSE OUTCOMES:

At the end of the program, the students will be able to:

CO1	Analyze and simulate discrete-time signals and systems in time, frequency, and z-domain using MATLAB.
CO2	Design and evaluate digital IIR and FIR filters using classical techniques and modern computational tools.
CO3	Implement signal processing algorithms on DSP hardware platforms (TMS320C6748 DSK) using Code Composer Studio for real-time applications.
CO4	Apply digital signal processing concepts to real-world applications such as image and audio processing and analyze performance metrics.

Course Description:

The Digital Signal Processing Laboratory provides students with practical exposure to the fundamental principles and implementation of digital signal processing using MATLAB, Code Composer Studio (CCS), and the TMS320C6748 DSP Starter Kit (DSK). The laboratory begins with the analysis of discrete-time signals and systems in the time, z-, and frequency domains, enabling students to understand the mathematical foundations of DSP through simulation. Students design and evaluate Infinite Impulse Response (IIR) and Finite Impulse Response (FIR) filters using standard digital filter design techniques and compare their performance based on frequency response and filter characteristics. The course further introduces real-time DSP programming using Code Composer Studio, where students implement DFT, FIR, and IIR algorithms on the TMS320C6748 DSP processor. Advanced experiments on image and audio/speech processing provide practical insight into real-world DSP applications. Through simulation and hardware implementation, the laboratory bridges theoretical concepts with real-time signal processing, equipping students with the technical skills required for applications in communication systems, multimedia, embedded systems, biomedical engineering, and intelligent signal processing.

Program Outcomes

PO1 (Engineering Knowledge): Apply knowledge of mathematics, natural science, computing, engineering fundamentals, and an engineering specialization to develop solutions for complex engineering problems.

PO2 (Problem Analysis): Identify, formulate, review research literature, and analyze complex engineering problems to reach substantiated conclusions with consideration for sustainable development.

PO3 (Design/Development of Solutions): Design creative solutions for complex engineering problems and develop systems, components, or processes that meet identified needs while considering public health and safety, whole-life cost, net-zero carbon, culture, society, and the environment.

PO4 (Conduct Investigations of Complex Problems): Use research-based knowledge and methods, including design of experiments, data analysis, interpretation, and synthesis, to investigate complex engineering problems and draw valid conclusions.

PO5 (Engineering Tool Usage): Create, select, adapt, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modeling, while understanding their limitations.

PO6 (The Engineer and the World): Analyze and evaluate societal, health, safety, legal, cultural, environmental, economic, and sustainability aspects relevant to engineering practice, and understand the associated responsibilities.

PO7 (Ethics): Apply ethical principles, demonstrate integrity, and commit to professional ethics, responsibilities, inclusivity, equity, and norms of engineering practice.

PO8 (Individual and Collaborative Teamwork): Function effectively as an individual and as a member or leader in diverse, multidisciplinary, and inclusive teams.

PO9 (Communication): Communicate effectively and inclusively on complex engineering activities with the engineering community and society through reports, documentation, presentations, and clear instructions.

PO10 (Project Management and Finance): Apply engineering and management principles to manage projects in multidisciplinary environments, considering financial, economic, and organizational aspects.

PO11 (Life-Long Learning): Recognize the need for and engage in independent, lifelong learning to adapt to technological advancements and changing professional environments.

Programme Specific Outcomes

Electronics & Communication Engineering Graduates will be able to:

PSO1. Apply the principles of electronics & computer engineering across diverse domains, including electronic design, software engineering, computer networks, embedded systems, operating systems, image processing, artificial intelligence, and associated areas.

PSO2. Analyze real-world challenges in electronics and the computer domain using the latest hardware and software tools to arrive at cost-effective and appropriate solutions.

PSO3. Create and validate pioneering methodologies in electronics and computer engineering to address societal challenges

. Course Outcomes (CO) to PO, PSO and BL Mapping

COs	At the end of this course, the student should be able to	Contact Hours	Marks	Program Outcomes (PO)	Program Specific Outcomes (PSO)	Bloom's Level (BL)
CO1	Analyze and simulate discrete-time signals and systems in time, frequency, and z-domain using MATLAB.	6	30	1, 2, 5,11	1	4
CO2	Design and evaluate digital IIR and FIR filters using classical techniques and modern computational tools.	4	20	1, 2, 5,11	1	4
CO3	Implement signal processing algorithms on DSP hardware platforms (TMS320C6748 DSK) using Code Composer Studio for real-time applications.	6	30	1, 2, 5, 8, 11	1	4
CO4	Apply digital signal processing concepts to real-world applications such as image and audio processing and analyze performance metrics.	4	20	1, 2, 5, 8, 11	1	3
	Total	**20**	**100**		**1**	

Course Articulation Matrix

COs / POs & PSOs	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
CO1	3	3			2						1	1		
CO2	3	3			2						1	1		
CO3	2	2			2			1			1	1		
CO4	2	1			2			1			1	1		
Average Articulation Level	**2.5**	**2.25**			**2**			**1**			**1**	**1**		

Lesson Plan

Exp. No	Topics	CO
-	Introduction to signal processing using MATLAB	
P1	Analysis of signals in time domain	CO1
P2	Analysis of signals and systems in z-domain	CO1
P3	Analysis of signals and systems in frequency domain	CO1
P4	IIR Filter Design	CO2
P5	FIR Filter Design	CO2
P6	Image processing	CO1
-	Introduction to Code Composer Studio (CCS)	
P7	DFT computation using TMS320C6748 DSP starter kit	CO3
P8	IIR Implementation using TMS320C6748 DSP starter kit	CO3
P9	FIR Implementation using TMS320C6748 DSP starter kit	CO3
P10	Audio/Speech processing using TMS320C6748 DSP starter kit	CO4

Justification Table for Course Articulation Matrix

CO	PO	Justification
CO1	PO1	Students apply the fundamentals of mathematics, signals and systems, and digital signal processing to analyze discrete-time signals and systems in the time, frequency, and z-domains.
	PO2	Students identify and analyze signal processing problems by interpreting the characteristics of discrete-time signals and systems using appropriate analytical methods.
	PO5	Students use modern engineering tools such as MATLAB for simulation, visualization, and analysis of DSP algorithms.
	PO11	Students complete laboratory experiments within the allotted time by planning simulations and managing available laboratory resources effectively.
	PSO1	Students develop domain-specific skills in signal analysis using modern computational tools relevant to electronics and communication engineering.
	PSO2	Students employ modern engineering tools to analyze communication signals, preparing them for communication engineering applications
CO2	PO1	Students apply DSP fundamentals and filter design principles to design digital IIR and FIR filters meeting specified design requirements.
	PO2	Students analyze filter characteristics such as passband, stopband, transition bandwidth, and frequency response to evaluate filter performance.
	PO5	Students employ MATLAB-based design and analysis tools to implement and evaluate digital filters.
	PO11	Students manage design parameters and computational resources while completing filter design experiments systematically.
	PSO1	Students gain practical competence in digital filter design, an essential component of modern communication and embedded signal processing systems.
	PSO2	Students employ modern engineering tools to analyze communication signals, preparing them for communication engineering applications
CO3	PO1	Students apply DSP concepts to implement signal processing algorithms on the TMS320C6748 DSP platform.
	PO2	Students analyze the implementation and performance of real-time DSP algorithms on embedded hardware platforms.
	PO5	Students utilize modern development tools such as Code Composer Studio and the TMS320C6748 DSP Starter Kit for real-time implementation.
	PO8	Students work individually and in laboratory teams to implement, debug, and validate DSP algorithms on the TMS320C6748 DSP Starter Kit using Code Composer Studio. They coordinate tasks, troubleshoot implementation issues collaboratively, and contribute effectively toward successful realtime DSP system development
	PO11	Students effectively manage laboratory hardware, software, and execution time while implementing DSP applications.
	PSO2	Students employ modern engineering tools to analyze communication signals, preparing them for communication engineering applications
CO4	PO1	Students apply digital signal processing principles to solve practical problems in image and audio signal processing.
	PO2	Students analyze the performance of DSP techniques by evaluating the outcomes of image and audio processing algorithms.
	PO5	Students employ modern software and DSP hardware tools to implement and assess real-world signal processing applications.
	PO8	Students work individually and in laboratory teams to implement, debug, and validate DSP algorithms on the TMS320C6748 DSP Starter Kit using Code Composer Studio. They coordinate tasks, troubleshoot implementation issues collaboratively, and contribute effectively toward successful realtime DSP system development
	PO11	Students organize experiments, analyze performance metrics, and complete application-oriented tasks within available laboratory resources.
	PSO2	Students employ modern engineering tools to analyze communication signals, preparing them for communication engineering applications

Evaluation Plan & Rubrics

1. Internal Assessment

- Internal Assessment Marks: 60 Marks (60%)
- The Weekly evaluation scheme is as follows:
    i. Conduction: 4M
    ii. Exercise: 2M
    iii. Lab Record Book: 4M

Attributes	Good [2]	Satisfactory [1]	Poor [0.5]
Experiment Execution [3]	Performs experiment accurately without assistance	Minor errors, little assistance needed	Unable to perform the experiment
Observation & Data Recording [3]	Complete and accurate observations	Minor omissions/errors	Incomplete records/No proper records
Results & Discussion [2]	Results presented accurately with proper justification	Results presented but no justification provided	Incomplete results and no justification provided
Rubrics to evaluate Exercise			
Exercise [2]	All questions are answered	Moderately answered	Not answered

Note: *Continuous evaluation of 10 experiments for 10 marks each will be evaluated, and 100 marks will be scaled down to 60 Marks.*

2. End-Semester Assessment

- End-semester assessment Marks: 40 marks (40%)
    i. Write up: 10M
    ii. Execution: 10M
    iii. Result: 5M
    iv. Viva voce: 15M

Rubrics to evaluate End-Sem Assessment			
Attributes	**Good**	**Satisfactory**	**Poor**
Write up [10]: [Aim, Circuit diagram / Program logic, Procedure, tabular columns / Flow chart, expected graphs/sample and formulas]	Everything is written properly in order [10-7]	Minimum mistakes in write-up [6-4]	More mistakes in write up [1-3]
Execution [15]	Connections are correct and conducted properly / program executed properly + calculations and results are correct [15]	Minimum mistakes in connections, conduction, calculation and results/program executed partially [6-14]	More mistakes in connections, conduction, calculation, and results/program not executed properly [2-5]
Results & Discussion [10]	Accurate result with proper interpretation [10]	Result obtained with minor errors [5-9]	Incorrect/no result [2-4]
Viva voce [5]	All questions are answered without mistake [5]	Moderately answered [2-4]	Not answered/partially answered [0-1]

COs	At the end of this course, the student should be able to	Internal Assessment	End-Sem Assessment	Total Marks	Program Outcomes (PO)	Program Specific Outcomes (PSO)
CO1	Analyze and simulate discrete-time signals and systems in time, frequency, and z-domain using MATLAB.	18	Write up: 10M (CO1/2/3/4) Execution: 10M (CO1/2/3/4) Result: 5M (CO1/2/3/4) Viva voce: 15M (CO1/2/3/4)	30	1, 2, 5,11	1,2
CO2	Design and evaluate digital IIR and FIR filters using classical techniques and modern computational tools.	12		20	1, 2, 5,11	1,2
CO3	Implement signal processing algorithms on DSP hardware platforms (TMS320C6748 DSK) using Code Composer Studio for real-time applications.	18		30	1, 2, 5,11	1,2
CO4	Apply digital signal processing concepts to real-world applications such as image and audio processing and analyze performance metrics.	12		20	1, 2, 5,11	1,2
	Total	**60**	**40**	**100**		**1**

Pre-lab Preparation

1. Each lab experiment requires preparation that may involve a significant amount of work.
2. The pre-lab work must be completed before arrival at the lab. The pre-lab work will count toward your lab grade and will be evaluated each week.
3. The student must revise concepts related to Signals and Systems and Digital Signal processing (DSP) before coming to the lab.

Lab Observation

1. Separate notebooks should be maintained for noting the observations made during the conduct of the lab.
2. All the calculations need to be done and get them verified by the faculty.
3. The corrected lab observation book will be the reference for writing up the record.

SAFETY PRECAUTIONS

Before Entering the Lab

- All the students are supposed to prepare the theory regarding the next experiment.
- Students are supposed to bring the lab copy.
- Previous programs should be written in the lab copy.
- All the students must follow the instructions, failing which he/she may not be allowed in the lab.

While Working in the Lab

- Adhere to the experimental schedule as instructed by the lab in-charge.
- Get the previous experiment result signed by the instructor.
- Get the output of the current experiment checked by the instructor in the lab copy.
- Each student should work on his/her assigned experiment kit at each turn of the lab.
- Take responsibility for valuable accessories.
- Concentrate on the assigned practical

LAB RULES (DO'S & DON'T)

Always:

- Enter the lab on time and leave at the proper time.
- Keep the bag in the respective place.
- Utilize lab hours in the corresponding.
- Turn off the instrumentation before leaving the lab unless a member of lab staff has specifically told you not to do so.
- Leave the labs at least as nice as you found them.
- If you notice a problem with a piece of equipment, please report it to lab staff immediately. Do not attempt to fix the problem yourself.

Never

- Don't damage the equipment.
- Don't bring any external material into the lab, except your lab record, a copy and books.
- Don't bring the mobile phones into the lab. If necessary, then keep them in silence mode.

Note: Please be considerate of those around you, especially in terms of noise level. While labs are a natural place for conversations of all types.

Signature
NBA Coordinator-SoEE

Seal & Signature
Associate Dean-SoEE

CO Wise Marks Distribution (Odd Page)

Exp. No	Exp name	CO1	CO2	CO3	CO4	Total
1	Analysis of signals in time domain					
2	Analysis of signals and systems in z-domain					
3	Analysis of signals and systems in frequency domain					
4	IIR Filter Design					
5	FIR Filter Design					
6	Image processing					
7	DFT computation using TMS320C6748 DSP starter kit					
8	IIR Implementation using TMS320C6748 DSP starter kit					
9	FIR Implementation using TMS320C6748 DSP starter kit					
10	Audio/Speech processing using TMS320C6748 DSP starter kit					
	Total					
	Average					

Total Marks (100)	
Marks reduced to 60	

Introduction to Signal Processing using MATLAB

MATLAB (Matrix Laboratory) is an interactive program for scientific and engineering numeric calculations, which has applications in almost all areas. In MATLAB each variable is a matrix. The fundamental operators (such as addition, multiplication) are programmed to deal with matrices when required. In the MATLAB environment, a matrix is a rectangular array of numbers. Special meaning is sometimes attached to 1-by-1 matrices, which are scalars, and to matrices with only one row or column, which are vectors.

Unlike languages such as C and C++, there is not much syntax in the MATLAB. Anyone who has knowledge of simple programming can work with the MATLAB. It is very rich with toolboxes, which consist of various functions depending on the applications. These functions can be used as a command to perform particular operations. Signal processing toolbox of MATLAB contains most of the functions for signal processing. By double clicking the MATLAB icon you can invoke the software. Figure 1 shows the command window.

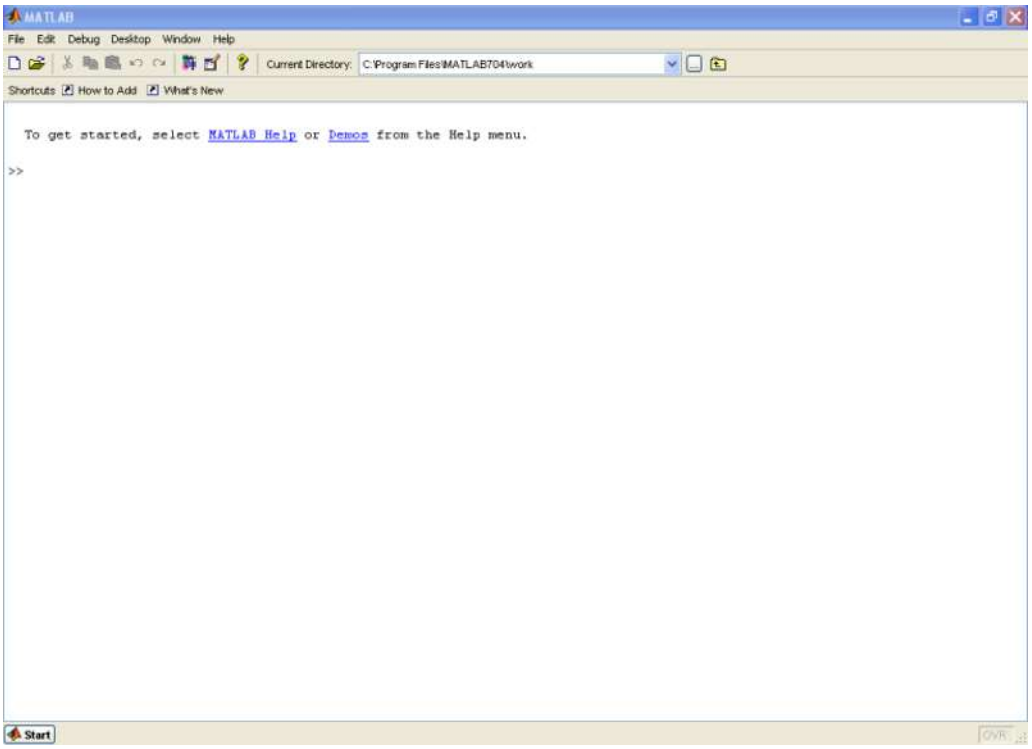


Fig. 1: MATLAB Command Window

Command window consists of prompt for entering the command for the execution of the function. Suppose we want to create a variable with a vector we can enter the following at the command prompt and press ENTER key.

```
>> a = [1 2 3 4 5]

a = 1 2 3 4 5
```

This will create a vector with variable name ‘a’.

If the command which is entered at prompt is terminated by the semicolon, then the values corresponding to the variable is not shown in the command window when you press the ENTER key. Similarly, any command can be entered at the prompt, as follows.

```
>> a=ones (2, 2)

a =

1        1

1        1
```

The above command will create a matrix of size 2x2 with all ones.

Similarly, there are many commands in MATLAB. In this lab the students shall get acquainted with the usage of these commands using some examples.

Execute the following commands from the command prompt and verify the outputs.

```
>> a  = [1  2  3   4]                          1×4 matrix
a =
      1       2       3       4
```

```
>> b  = a + 2                                  adds 2 to every element of "a"
                                               matrix
b =
      3       4       5       6
```

```
>> c  = a + b                                  adds the two matrices
c =
      4       6       8      10
```

```
>> d  = b – a                                  matrix subtraction
d =
      2       2       2       2
```

```
>> e  = a*2                                    multiplying by a scalar
e =
      2       4       6       8
```

```
>> A  = [1 2 3 ;   4   5  6 ; 7 8 9]          3×3 matrix
A =
      1       2       3
      4       5       6
      7       8       9
```

```
>> B = A'                                      transpose of matrix

B =

      1       4       7

      2       5       8

      3       6       9
```

```
>> C = A.*B                                  element by element multiplication
C =

     1     8    21

     8    25    48

    21    48    81
```

```
>> C = A * B                                 matrix multiplication
C =

    14    32    50

    32    77   122

    50   122   194
```

```
>> D = B./A                                  element by element division
D =
    1.0000    2.0000    2.3333
    0.5000    1.0000    1.3333
    0.4286    0.7500    1.0000
```

```
>> E=A.^2                                    element by element squaring
E =

     1     4     9

    16    25    36

    49    64    81
```

```
>> F = A^2                                   square of the matrix
A×A F =
    30    36    42
    66    81    96
   102   126   150
```

```
>> A(6)                                      extracting the 6th element of the matrix
ans =

     8
```

```
>> A(2,:)                                    extracting the entire 2nd row of the matrix
ans =

     4     5     6
```

```
>> A(:,2)                                  extracting the entire 2nd column of the matrix
ans =

     2

     5

     8
```

```
>> G = [A;B]                               joining the two matrices

G =
     1     2     3
     4     5     6
     7     8     9
     1     4     7
     2     5     8
     3     6     9
```

```
>> [r c] = size(G)                         extracting the size of the matrix

r =

     6

c =

     3
```

```
>> X = ones(3,3)                           matrix with all "1"

X =

     1     1     1

     1     1     1

     1     1     1
```

```
>> Y = zeros(3,3)                          matrix with all "0"

Y =

     0     0     0

     0     0     0

     0     0     0
```

```
>> R = rand(3,3)                           random numbers between 0 & 1
R =
    0.8147    0.9134    0.2785
    0.9058    0.6324    0.5469
    0.1270    0.0975    0.9575
```

```
>> R2 = randn(3,3)                    normally distributed random numbers
R2 =
    2.7694    0.7254   -0.2050
   -1.3499   -0.0631   -0.1241
    3.0349    0.7147    1.4897

>> n = 0:9                            integers from 0 to 9
n =
    0    1    2    3    4    5    6    7    8    9

>> t = 0:0.1:1                        values from 0 to 1 in steps of 0.1
t =
         0    0.1000    0.2000    0.3000    0.4000    0.5000    0.6000    0.7000
    0.8000    0.9000    1.0000

>> x = 5 + 3i          complex valued variable
x =
   5.0000 + 3.0000
>> real(x)                  real part of x
ans =
    5

>> imag(x)           imaginary part of x
ans =
    3

>> abs(x)                magnitude of x
ans =
   5.8310

>> angle(x)           angle of x in radians
ans =
    0.5404
```

Information about any command/function can be obtained by typing “help” followed by the command.

>> help clc

clc Clear command window.

clc clears the command window and homes the cursor.

>> help conv

conv Convolution and polynomial multiplication.

C = conv(A, B) convolves vectors A and B.

>> help plot

plot Linear plot.

plot(X,Y) plots vector Y versus vector X

>> help stem

stem Plot discrete sequence data

stem(X,Y) plots the data sequence Y, at values specified by X.

M-Files

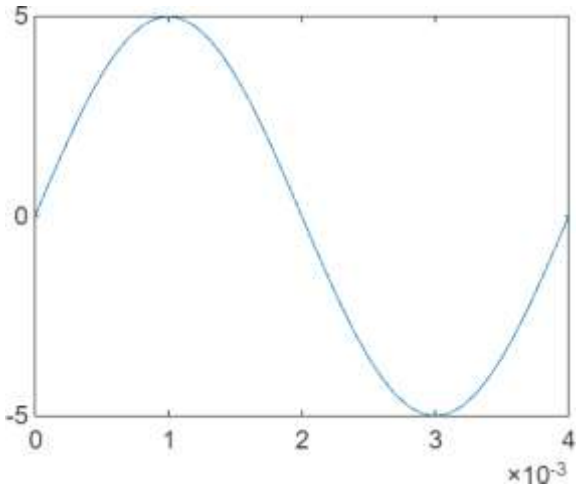
MATLAB programs can be written using the Editor and saved as M-files. Use the MATLAB Editor to create a file containing the same statements you would type at the MATLAB command line. Save the file under a name that ends with “.m”.

Let us learn this by writing a simple program for generating a sine wave and plotting it using plot command.

In the File menu select the New option and then select M-file. MATLAB editor window will open. Then enter the following program. Then save the program in a “.m” file and execute (Run) the program.

```
clc;                % clears the command window
clear all;          % clears all the variables
close all;          % closes all existing figures
Fs = 10000;         % sampling frequency 10 kHz
f = 250;            % signal frequency 250 Hz
t = 0:(1/Fs):(1/f);         % time sequence for one period at intervals of T=1/Fs
```

```
x = 5*sin(2*pi*f*t);        % sinewave of amplitude 5 and frequency f
plot(t,x);                  % plot the sinewave
```



NOTE: While saving the program, give meaningful filename. Strictly do not give a filename which is already a MATLAB command. This will overwrite the existing command file which is a built-in function.

Comments can be written in the program after the "%" symbol.

The program can be executed by selecting the Run option from the Debug menu. The output is plotted in separate window which is shown above.

Experiment 1: Analysis of Signals in time domain

1. **Aim:** To study the analysis of signals in time-domain and understand the concepts of sampling, convolution and correlation.

2. **Software used:** MATLAB

3. **Theory:**

Time-domain analysis is the study of a signal by observing how its amplitude varies with time. It is one of the fundamental approaches in digital signal processing and provides insight into the behavior and characteristics of signals before they are analyzed in other domains. Most real-world signals, such as speech, audio, and biomedical signals, are initially examined in the time domain to understand their temporal properties.

Digital signal processing operates on discrete-time signals, which are obtained by sampling continuous-time signals at regular intervals. Sampling is the process of converting an analog signal into a sequence of discrete samples while preserving the essential information of the original signal. Proper selection of the sampling frequency is necessary to accurately reconstruct the original signal and avoid information loss.

Convolution is a fundamental mathematical operation used to determine the output of a linear time-invariant (LTI) system for a given input signal and system impulse response. It describes how the characteristics of a system modify the input signal. Correlation is another important operation that measures the degree of similarity between two signals or between different segments of the same signal. It is widely used in applications such as pattern recognition, synchronization, signal detection, and communication systems.

Understanding these concepts is essential for analyzing and processing digital signals and forms the foundation for advanced topics such as digital filter design, frequency-domain analysis, and real-time signal processing.

4. **Program:**

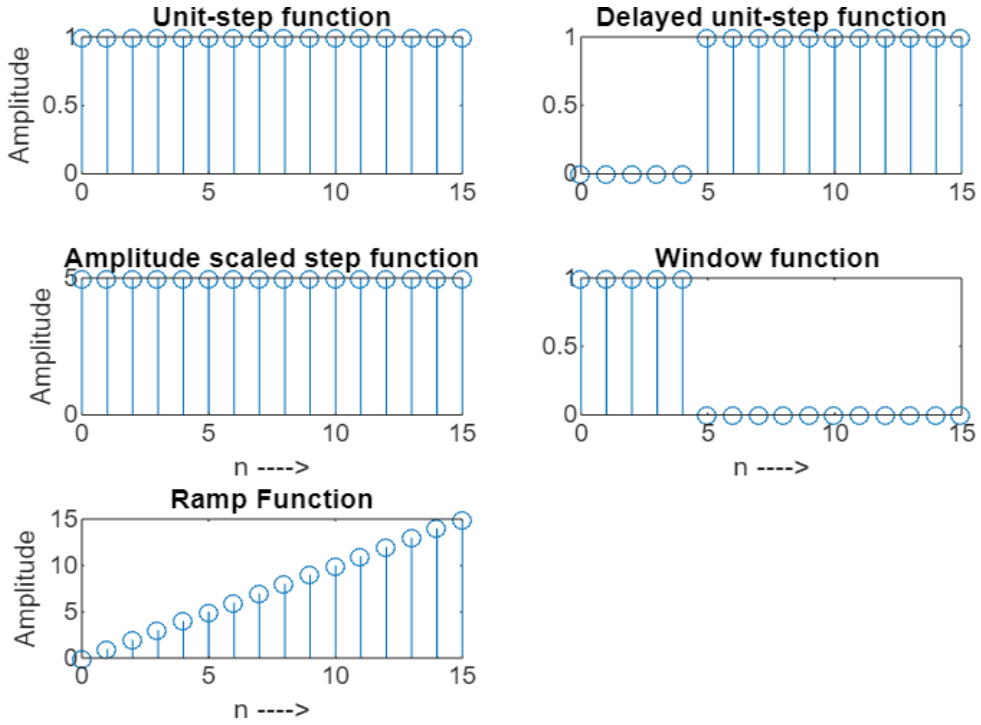
Example-1: Generate and plot the following signals:

a) Unit-step function b) Delayed step function with a delay of 5.

c) Step function with amplitude 5 d) Rectangular window of length 5 e) Ramp signal.

```
clc;                % clears the command window
clear all;          % clears all variables
close all;          % closes all the figure windows
n=0:15;             % time index for the discrete signal from 0 to 15
u=[(n)>=0];         % Unit Step
u5=[(n-5)>=0];      % Delayed step
a5=5*u;             % Scaled unit step
xn=u-u5;            % rectangular window x[n]= u[n]-u[n-5]
```

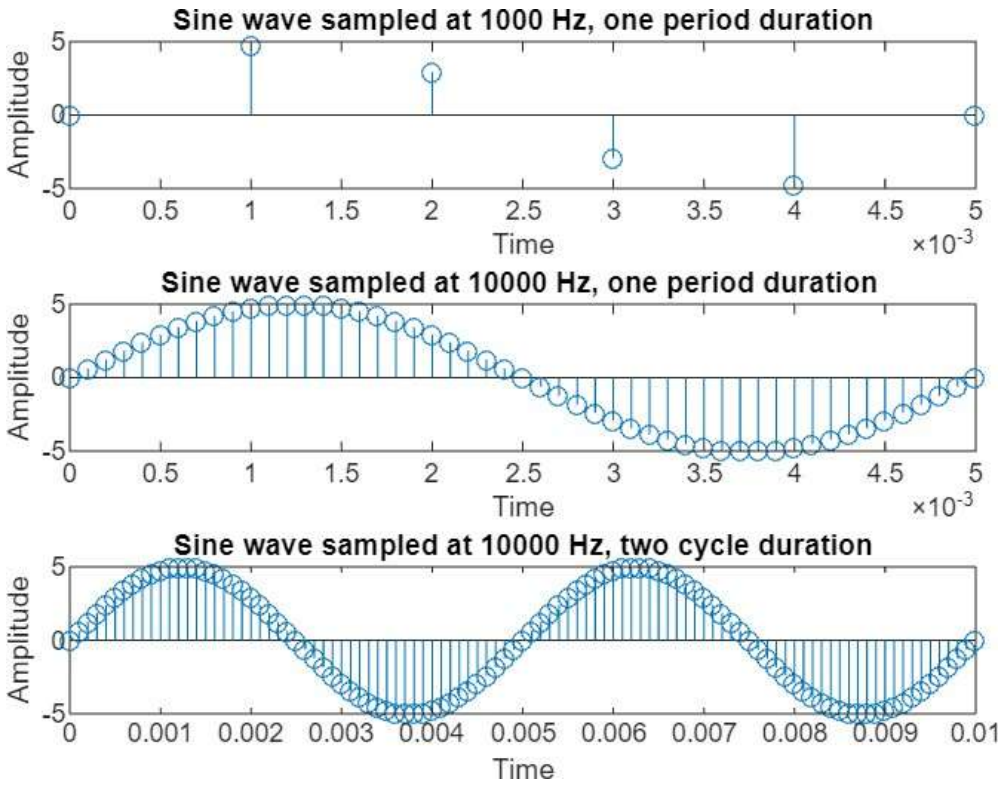
```
xr=n.*u;            % Ramp signal
subplot(321); stem(n,u); title('Unit-step function');
ylabel('Amplitude'); subplot(322); stem(n,u5);
title('Delayed unit-step function'); subplot(323);
stem(n,a5); title('Amplitude scaled step function');
ylabel('Amplitude'); xlabel('n ->');
subplot(324); stem(n,xn); title('Window function '); xlabel('n     >');
subplot(325); stem(n,xr);
title('Ramp Function');
ylabel('Amplitude'); xlabel('n ->');
```



Example-2: Write MATLAB code to generate sine waves at different sampling frequencies.

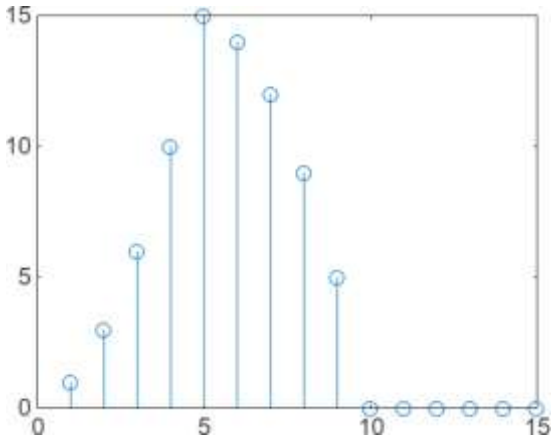
```
clc; clear all; close all;
Fs=1000;                        % sampling frequency 1000 Hz
f=200;                          % signal frequency 200 Hz
t=0:1/Fs:(1/f);    % generation of the time sequence for 1 period duration.
x=5*sin(2*pi*f*t);     %sine wave of freq 200 Hz and amplitude 5, sampled
at Fs
subplot(3,1,1); stem(t,x);
title('Sine wave sampled at 1000 Hz, one period duration');
ylabel('Amplitude'); xlabel('Time');
Fs1=10000;      % sampling frequency 10000 Hz
t=0:1/Fs1:(1/f);
```

```
x=5*sin(2*pi*f*t);
subplot(3,1,2); stem(t,x);
title('Sine wave sampled at 10000 Hz, one period duration');
ylabel('Amplitude'); xlabel('Time')
t=0:1/Fs1:(2/f);      % time sequence for two cycle duration
x = 5*sin(2*pi*f*t);
subplot(3,1,3);
stem(t,x);
title('Sine wave sampled at 10000 Hz, two cycle duration');
ylabel('Amplitude'); xlabel('Time');
```



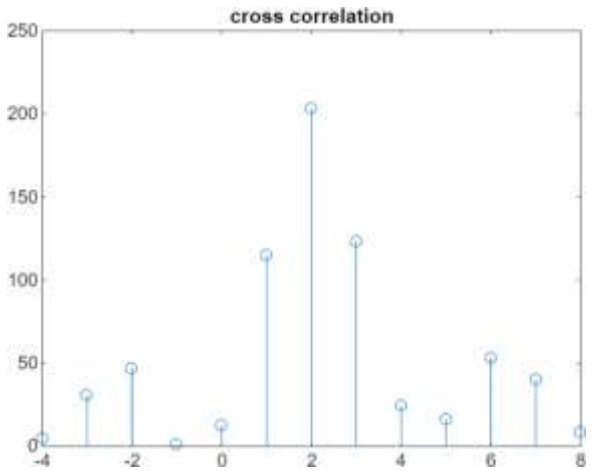
Example-3: Write MATLAB program to find and plot the convolution between two signals x[n] = u[n] – u[n−5] and y[n] = [1,2,3,4,5], where u[n] is a unit step function in the range 0:10. Use “conv” function.

```
clc; clear; close all;
n=0:10;
u=[n>=0];
u5=[(n-5)>=0];
x=u-u5;
y=[1,2,3,4,5];
z=conv(x,y);     % performs convolution of x & y
stem(n,z);
```



Example-4: Write a MATLAB program to find and plot the cross-correlation between the two signals x[n] = {3,11,7, 0, −1,4,2} and y[n] = x[n−2] – w[n], where w[n] is a Gaussian sequence with zero mean and unit variance.

```
clc; clear; close all;
x=[3,11,7,0,-1,4,2];  % first sequence
nx=-3:3;              % index for signal x
ny=nx+2;              % index for signal x delayed by 2
w=randn(1,length(x)); % random sequence generation
y=x+w;                % second sequence
ns=nx(1)+ny(1);       % starting index
ne=nx(length(x))+ny(length(y)); % ending index
nz=ns:ne;             % index for correlation output
z=xcorr(x,y);         % cross correlation
stem(nz,z);
title('cross correlation');
```



The signal x and its delayed signal y have maximum correlation at time n = 2. Hence, the correlation plot shows a maximum peak at n = 2. Such signal processing is used in Radar applications.

5. Exercise

1. Generate the following signals
   (a) $(-1/2)^k$, $(2)^k$ and $(-2)^k$, assuming k = 0 to 15
   (b) Square wave with duty cycle 25%, 50% and 75%. (Use *square* function)
   (c) Triangular wave. (Use *sawtooth* function)
   (d) A multi-tone signal having the frequencies 10Hz, 30Hz and 60Hz.

2. Write a MATLAB program to sketch the following discrete-time signals in the time range of -10 to +10. If the sequence is complex, plot the magnitude and angle separately.

   (a) x[n] = u[n] − u[n − 3]

   (b) x[n] = sin (nπ/3) $u[n]$

   (c) x[n] = $0.5^{n}\ e^{j\pi n/2}$

3. A simple digital differentiator is given by y[n] = x[n] − x[n − 1]. Implement this differentiator and test with the following sequences. Plot the results and comment on the appropriateness of this differentiator.

   (a) x[n] = 5(u[n] − u[n − 20])

   (b) x[n] = (u[n] − u[n − 10]) + (20 − n)(u[n − 10] − [n − 20])

   (c) x[n] = sin(πn/25)(u[n] − u[n − 100]))

4. Write a MATLAB program to generate a pure sine wave of frequency 80Hz and add additive white Gaussian noise with SNR of 10dB. (Use the function awgn.) Find and plot the autocorrelation of the resulting sequence.

Experiment Execution [3]	Observation & Data Recording [3]	Results, Discussion and interpretation [2]	Exercise [2]	Total [10]

Experiment 2: Analysis of Signals and Systems in Z-domain

1. **Aim:** To study the analysis of signals and systems in Z-domain

2. **Software used:** MATLAB

3. **Theory:**

The z-transform is a fundamental mathematical tool used in digital signal processing to analyze discrete-time signals and systems in the complex frequency domain. It provides a convenient method for representing and solving linear difference equations, determining system characteristics, and analyzing the stability and frequency response of discrete-time systems. The z-transform of a discrete-time signal x[n] is defined as

$$X(z) = \sum_{n=-\infty}^{\infty} x[n]\, z^{-n},$$

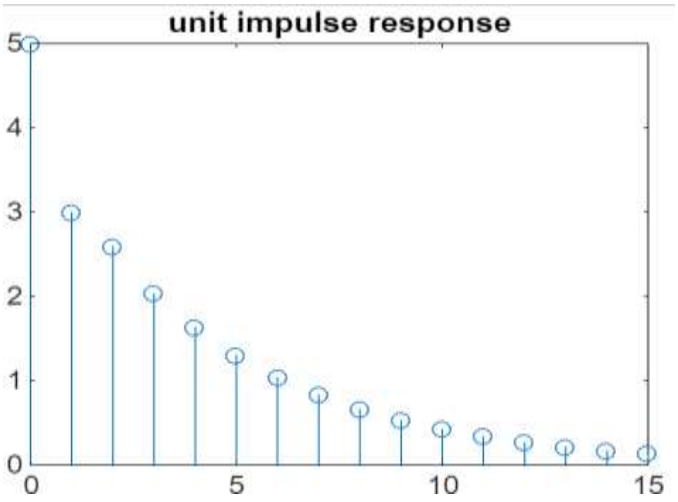
where z is a complex variable. The set of values of z for which the z-transform converges is known as the Region of Convergence (ROC), which plays an important role in determining the existence, causality, and stability of a system.

The transfer function of a discrete-time system is expressed as the ratio of the output to the input in the z-domain and is represented as $H(z) = \frac{Y(z)}{X(z)}$. The locations of poles and zeros of the transfer function provide valuable information about the system's behavior. A system is considered stable if all its poles lie inside the unit circle in the z-plane, while the frequency response of the system can be obtained by evaluating the transfer function on the unit circle. The z-transform simplifies the analysis of discrete-time systems by converting convolution operations in the time domain into multiplication in the z-domain, making it an indispensable tool in the design and analysis of digital filters and other DSP applications.

4. **Program:**

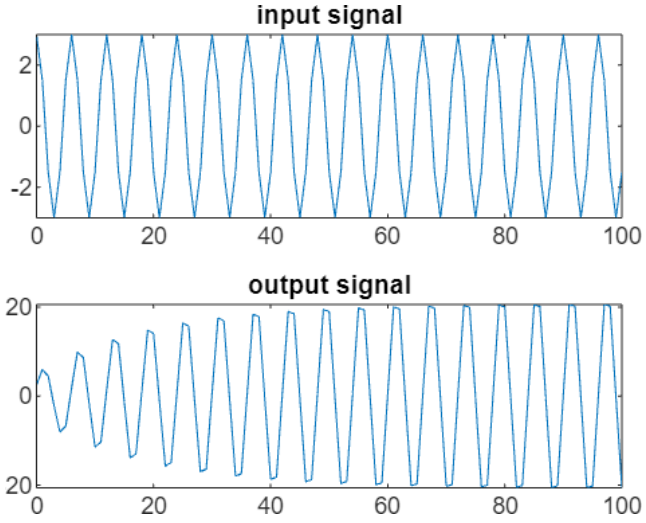
Example-1: Determine the unit impulse response h[n] of a system described by the difference equation: y[n] − 0.6y[n – 1] − 0.16y[n– 2] = 5x[n].

```
clc; clear; close all;
a=[1,-0.6,-0.16];      % denominator coefficients of system function
b=[5,0,0];             % numerator coefficients of system function
N=16;                  % length of the output
x=[1,zeros(1,N-1)];    % input is impulse function of length N
h=filter(b,a,x);       % output of system is impulse response n=0:N-
1;
stem(n,h);
title ('unit impulse response');
```



Example-2: Consider a causal LTI system described by the difference equation $y(n) = 1/3\,[x(n) + x(n-1) + x(n-2)] + 0.95y(n-1) - 0.9025y(n-2)$ with initial conditions $y(-1) = -2$, $y(-2) = -3$; $x(-1) = 1$, $x(-2) = 1$. Write a MATLAB program to determine the response of the system for the input $x(n) = \cos(n\pi/3)u(n)$.

```
clc; clear; close all;
a = [1 -0.95 0.9025];
b = [1 1 1]/3;
yi = [-2 -3];              % Initial conditions array
xi = [1 1];                % Initial conditions array
ic = filtic(b,a,yi,xi);   % converts past input xi and past output yi into
% initial conditions which is required for the computation of output
n = 0:100;
u = [n>=0];
x = 3*cos(n*pi/3).*u;     % input signal
y = filter(b,a,x,ic);     % o/p signal
subplot(211), plot(n,x); title('input
signal') subplot(212), plot(n,y);
title('output signal')
```



Example-3: Consider a causal LTI system described by the difference equation given by y(n) – 1.81y(n - 1) + 0.81y(n – 2) = x(n) + 0.5 x(n – 1). Write MATLAB program to obtain the following:

a) System function H(z) in the partial fraction form

b) Pole – zero plot

c) Frequency response of the system

d) Unit impulse response of the system

e) Output of the system for the given input x[n] = 3cos(nπ/3)u[n]

```
clc; clear; close all; a=[1,-
1.81,0.81];
b=[1,0.5,0];

% To obtain system function in the partial fraction form
[r p k]=residuez(b,a); % residues, poles and direct terms of the partial
fraction % expansion of the system function.
disp(['r = ' num2str(r')]);
disp(['p = ' num2str(p')]);
disp(['k = ' num2str(k')]);

% To obtain the pole-zero plot
[z p k]=tf2zp(b,a); % To obtain zeros, ploes and gain constant
of the system. disp('Zeros are at'); disp(z);

disp('poles are at'); disp(p);
disp('Gain Constant is'); disp(k);
zplane(b,a); % pole-zero plot

% To obtain frequency response of the system

[h w]=freqz(b,a); % returns the N-point complex frequency response
figure;

subplot(211), plot(w,abs(h)), title('Magnitude
Response'); xlabel('Frequency in radians'),
ylabel('Amplitude'); subplot(212),
plot(w,angle(h)), title('Phase Response');

xlabel('Frequency in radians'), ylabel('Phase in radians');

% To Obtain the unit impulse response of the system
L=15; % First 15 values of the impulse response
[h n]=impz(b,a,L);
figure; stem(h);
title('Impulse Response'); grid on;
xlabel('Sample Number'),
ylabel('Amplitude');

% To find the output of the system for the given input
n=0:100;
u=[n>=0];
x=3*cos(n*pi/3).*u; % input signal
y=filter(b, a, x); % o/p signal figure;
```

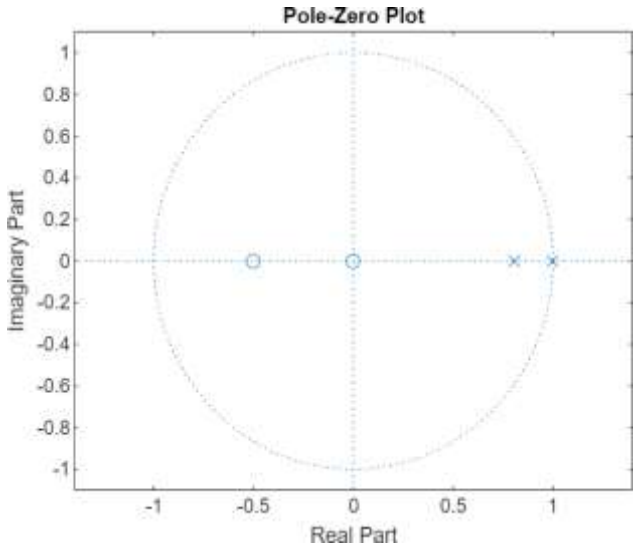
```
subplot(211), plot(n,x);
subplot(212), plot(n,y);
```

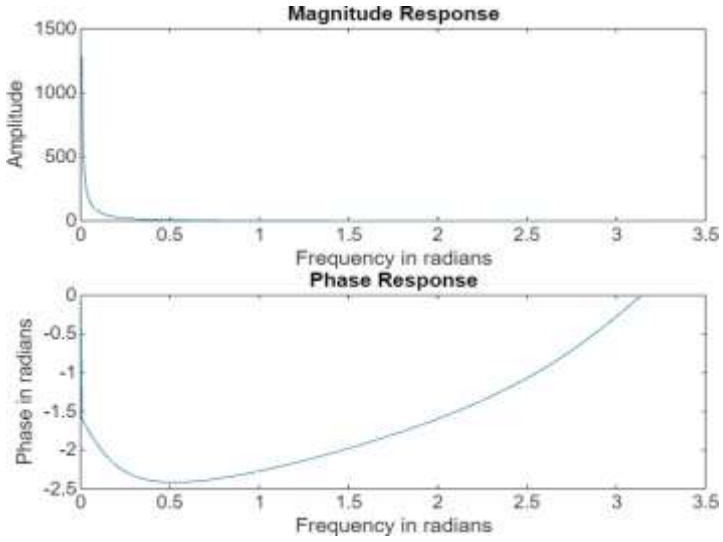
Results:
r = 7.8947 -6.8947
p = 1 0.81
k = 0

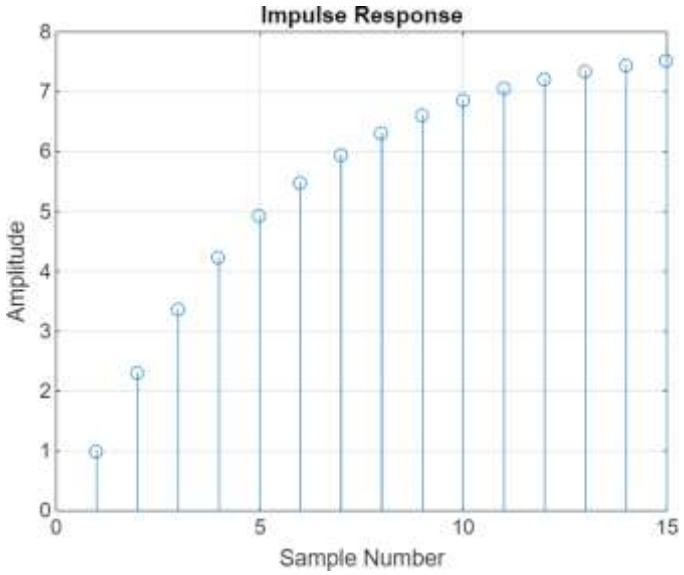
Zeros are at
0
-0.5000

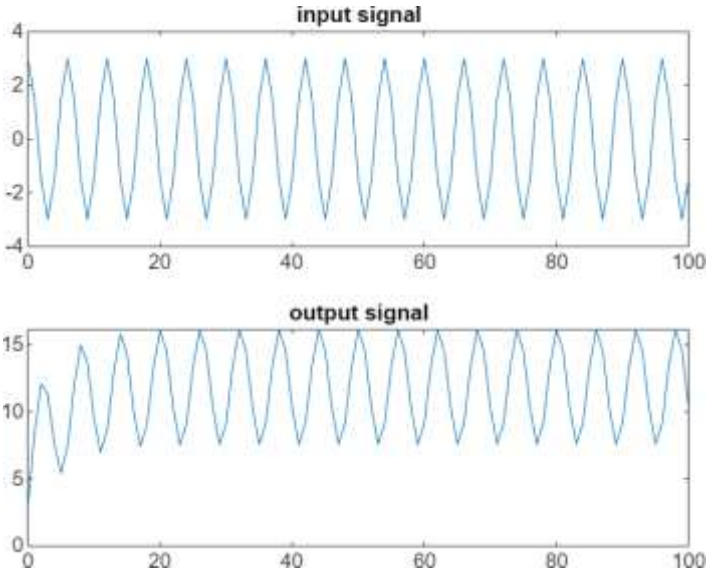
poles are at
1.0000
0.8100

Gain Constant is
1




Magnitude Response
Amplitude
Frequency in radians
Phase Response
Phase in radians
Frequency in radians


Impulse Response
Amplitude
Sample Number


input signal
output signal

5. Exercises

1. Find the system output y[n]; 0≤ n ≤10 of an LTI system with impulse response:
   h[n] = $(0.5)^n\{u[n] - u[n-10]$) when the inputs are:
   (a) $x[n] = (0.8)^n\ (u[n] - u[n-5])$
   (b) x[n] = δ[n] + 3δ[n – 1] + 4δ[n – 3]

2. An anti-causal LTI system is described by the system function $H(z) = (z^2 - 1) / (z - 3)^2$. Determine the following using MATLAB

   a) impulse response

   b) difference equation representation

   c) pole-zero plot

   d) output y[n] if the input is $x(n) = 3\sin(n\pi/3)\, u(n)$

3. Write a MATLAB program to determine the system function of a system for which the location of zeros and poles, and the value of the gain factor are specified. Test this program for various specifications.

4. Consider a causal LTI system described by the difference equation $y[n] = x[n] - x[n-1] + 0.81y[n-2]$ with initial conditions $y(-1) = y(-3) = 2$ Write a MATLAB program to determine the response of the system for the input $x(n) = 0.7^n u[n+1]$.

Experiment Execution [3]	Observation & Data Recording [3]	Results, Discussion and interpretation [2]	Exercise [2]	Total [10]

Experiment 3: Analysis of Signals and Systems in Frequency domain

1. **Aim:** To study the frequency-domain representation and analysis of signals and systems.

2. **Software used:** MATLAB

3. **Theory:**

Frequency-domain analysis is a fundamental technique in digital signal processing used to examine the frequency components present in a signal. While time-domain analysis describes how a signal varies with time, frequency-domain analysis reveals how the signal's energy or power is distributed across different frequencies. This representation is particularly useful for understanding the spectral characteristics of signals and the behavior of digital systems.

The transformation from the time domain to the frequency domain is achieved using the Discrete Fourier Transform (DFT), which expresses a finite-length discrete-time signal as a sum of sinusoidal components of different frequencies. The Fast Fourier Transform (FFT) is an efficient algorithm for computing the DFT, significantly reducing the computational complexity. The magnitude spectrum indicates the strength of various frequency components, while the phase spectrum provides information about their phase relationships.

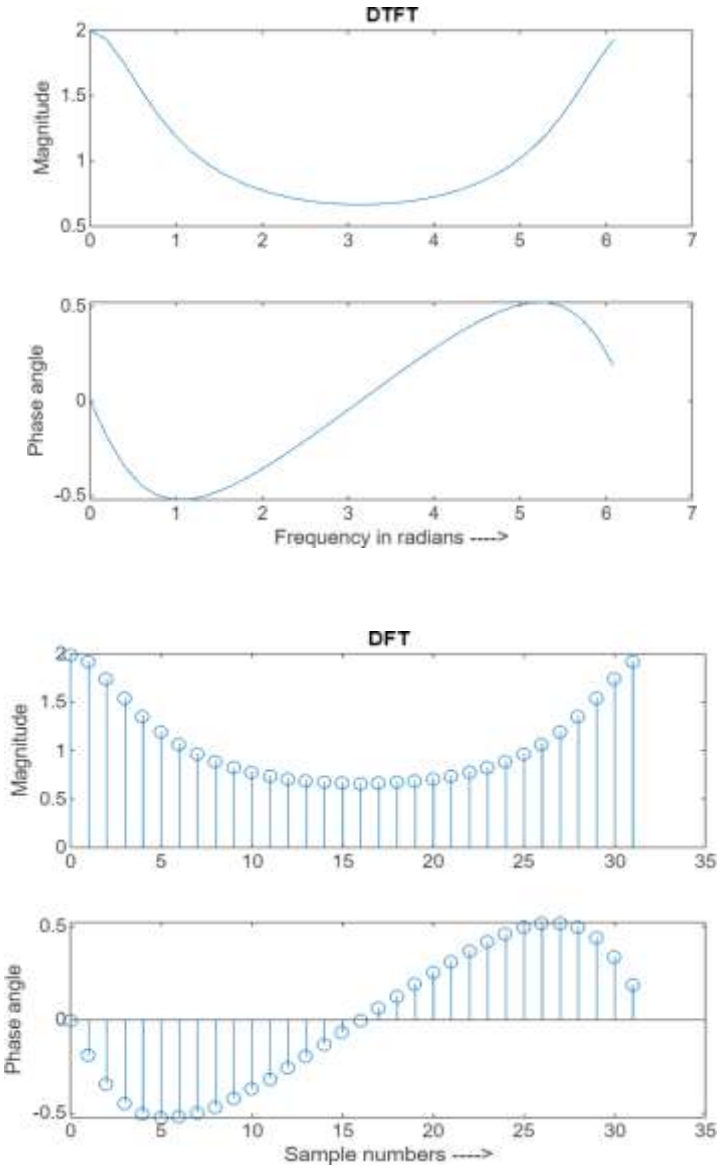
The frequency response of a discrete-time system describes its effect on different frequency components of the input signal. It is obtained by evaluating the system's transfer function on the unit circle in the z-plane. Frequency-domain analysis is extensively used in digital filter design, spectrum analysis, communication systems, speech and image processing, biomedical signal analysis, and many other signal processing applications. A clear understanding of the frequency-domain representation of signals and systems is essential for analyzing, designing, and implementing efficient digital signal processing algorithms.

4. **Program:**

Example-1: Find the Fourier transform (DTFT) and Discrete Fourier transform (DFT) of the signal $x[n] = (0.5)^n u[n]$, and plot the magnitude and the phase of the same.

```
clc; clear all; close all;
N=32; n=0:(N-1); w=n*2*pi/N;
x=(0.5).^n;
X=fft(x);% compute Fourier transform of the signal
subplot(211); plot(w,abs(X)); ylabel('Magnitude');
title('DTFT');
subplot(212); plot(w,angle(X));
ylabel('Phase angle'); xlabel('Frequency in radians >')
```

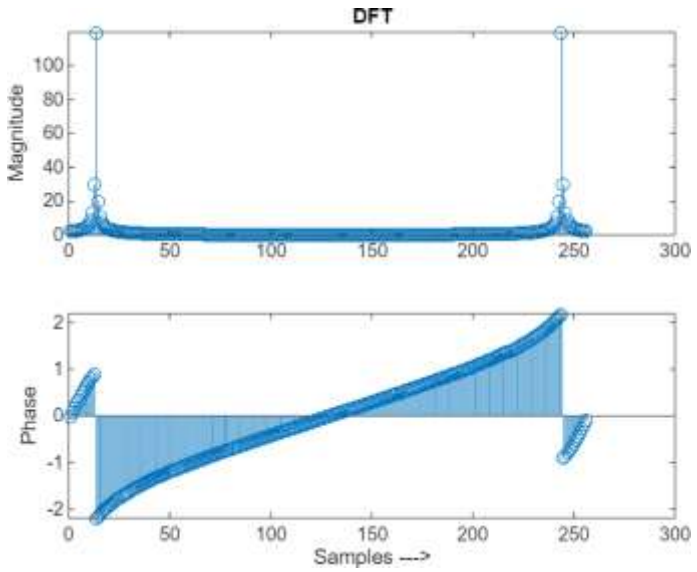
```
figure; subplot(211); stem(n,abs(X)); ylabel('Magnitude');
title('DFT'); subplot(212); stem(n,angle(X)); ylabel('Phase angle');
xlabel('Sample numbers ---- ');
```



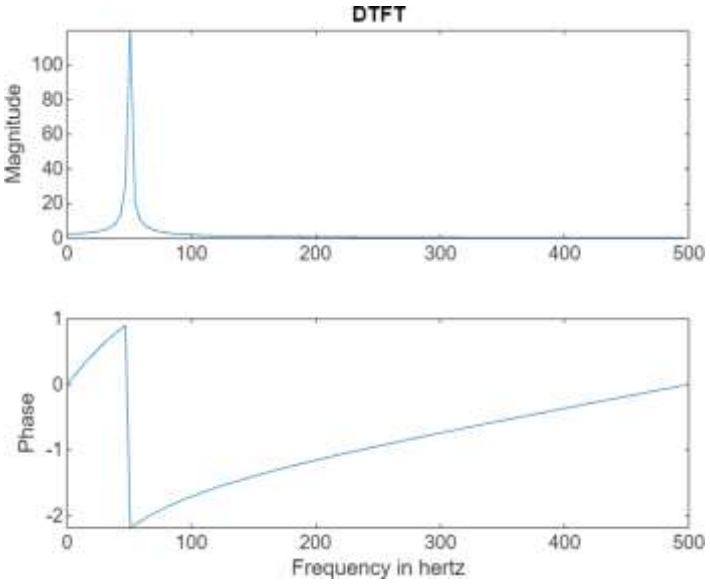
Note that DTFT is a continuous function of ω and hence “plot” command is used to plot the graphs. Only one period of DTFT has been shown (from 0 to 2π radians). DFT is a discrete function and hence “stem” command is used to plot the graphs. The x-axis is sample numbers.

Example-2: Find the DFT and DTFT of 20 cycles of a sinewave of frequency 50 Hz, sampled at 1000 Hz. Plot the magnitude and phase.

```
clc; clear all; close all; f =
50; Fs = 1000;
t = 0:(1/Fs):(20/f);          % 20 cycles of input x =
sin(2*pi*f*t);
X = fft(x,256);               % 256-point DFT
subplot(211); stem(abs(X));
ylabel('Magnitude');
title('DFT'); subplot(212);
stem(angle(X));
ylabel('Phase');
xlabel('Samples --->')
L = length(X)/2;              % take half of the samples (0 to pi radians) f
= (0:(L-1))/L*(Fs/2);         % convert sample numbers into frequency in Hz
figure; subplot(211); plot(f,abs(X(1:L)));
ylabel('Magnitude');
title('DTFT'); subplot(212);
plot(f,angle(X(1:L)));
ylabel('Phase'); xlabel('Frequency
in hertz');
```



The x-axis ranges from 0 to 255 corresponding to 0 to 2π radians



Note: Since the sampling frequency is 1000 Hz, the highest frequency component in the graph should be 500 Hz, corresponding to π radians. Hence half of the samples of DFT are taken for plotting. The plot shows a peak at 50 Hz which is the frequency of the sinewave signal.

Example-3: Write a MATLAB program to compute the circular convolution of two signals x1[n] = {1, 2, 3, 4, 5, 6, 7, 8} and x2[n] = {1, 1, 1, 1} using DFT-IDFT method, and verify it using direct computation. Also show how linear convolution can be achieved using DFT-IDFT method.

```
clc; clear all; close all; x1 =
[1 2 3 4 5 6 7 8];
x2 = [1 1 1 1];
N1 = length(x1);          % length of the first sequence N2
= length(x2);             % length of the second sequence N =
max(N1,N2);               % N is the maximum of N1 & N2
y = cconv(x1,x2,N);       % direct computation of N-point circular convolution

% DFT-IDFT method
X1 = fft(x1,N);
X2 = fft(x2,N);
Y = X1.*X2;               % element by element multiplication of two DFTs
z = ifft(Y,N);            % inverse DFT of the product gives circ. convolution
display(y); display(z);

% Linear convolution using the above method
yy = conv(x1,x2);         % direct computation of linear convolution N
= length(yy);             % Length N = N1 + N2 - 1
X1 = fft(x1,N);
X2 = fft(x2,N);
Y = X1.*X2;
```

```
zz = ifft(Y,N);          % Linear convolution using DFT-IDFT
display(yy); display(zz);
```

Results:

```
y =
    22    18    14    10    14    18    22    26 (circular convolution using direct method)

z =
    22    18    14    10    14    18    22    26 (circ. conv. using DFT-IDFT method)

yy =
     1     3     6    10    14    18    22    26    21    15     8 (linear convolution direct)

zz =
     1     3     6    10    14    18    22    26    21    15     8 (lin. conv. DFT-IDFT method)
```

5. Exercises:

1. Let x[n] = (0.9 $e^{j3\pi n}$), $0 \leq n \leq 15$. Determine X($e^{j\omega}$) and verify its periodicity property.

2. Determine the frequency response of the filters, which are represented by the difference equations given below. Also, state what type of filter each defines by observing the response (like LPF, HPF etc.)
   a) $y[n] + 0.13y[n-1] + 0.52y[n-2] + 0.3y[n-3] = 0.16x[n] - 0.48x[n-1] + 0.48x(n-2) - 0.16x(n-3]$
   b) $y[n] - 0.268y[n-2] = 0.634x[n] - 0.634x[n-2]$
   c) $y[n] + 0.268y[n-2] = 0.634x[n] + 0.634x[n-2]$
   d) $10y[n] - 5y[n-1] + y[n-2] = x[n] - 5x[n-1] + 10x[n-2]$

3. Find and plot the frequency response of a causal 3-point moving averager. Generate the first 100 samples of 0.9u[n] and corrupt it using a random signal. Pass the corrupted signal through the moving point averager and plot the output.

4. Find the DFT of multitone sinewave with 30Hz, 80Hz, 120Hz frequency components and plot the magnitude Vs. frequency in Hz.

Experiment Execution [3]	Observation & Data Recording [3]	Results, Discussion and interpretation [2]	Exercise [2]	Total [10]

Experiment 4: IIR Filter Design

1. **Aim:** To design and analyze analog and digital infinite-duration impulse response (IIR) filters.

2. **Software used:** MATLAB

3. **Theory:**

An Infinite Impulse Response (IIR) filter is a recursive system whose output depends on the present and past input samples as well as past output samples. Because of this feedback structure, its impulse response theoretically continues for an infinite duration. IIR filters generally achieve the required frequency-selective characteristics with a lower filter order than Finite Impulse Response (FIR) filters, making them computationally efficient.

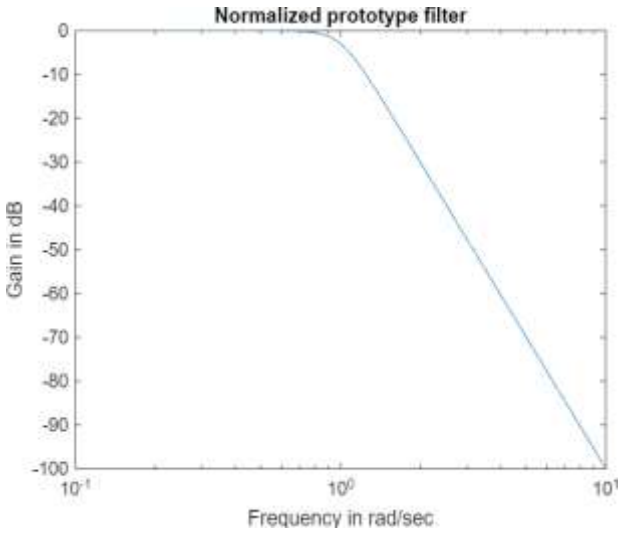
Analog IIR filters are commonly designed using standard approximations such as Butterworth, Chebyshev Type I, Chebyshev Type II, and Elliptic responses. These approximations differ in passband flatness, stopband attenuation, transition width, and ripple characteristics. Digital IIR filters are usually obtained from analog prototypes using transformation techniques such as the impulse invariance method or bilinear transformation.

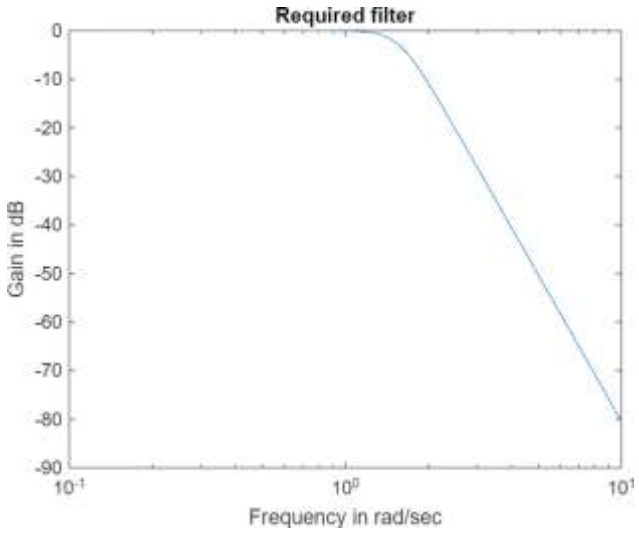
IR filters are widely used in audio processing, communication systems, biomedical signal processing, control systems, and noise reduction applications.

4. **Program:**

Example-1: Design an analog Butterworth low-pass prototype filter of order 5. Use it to get an unnormalized actual Butterworth filter with cutoff frequency of 0.5π rad/sec.

```
clc; clear; close all;
N = 5;                          % order of the filter
[z p k] = buttap(5);            % Butterworth LPF prototype
[num, den] = zp2tf(z,p,k);      % pole-zero to transfer function
[h,w] = freqs(num,den);         % frequency response in s-domain
semilogx(w,20*log10(abs(h)));   % plot in semi log graph
title('Normalized prototype filter');
xlabel('Frequency in rad/sec'); ylabel('Gain in dB');
wc = 0.5*pi;                    % actual cutoff frequency
p1 = p*wc;                      % scale the poles
k1 = k*wc^N;                    % scale the gain factor
[num1,den1] = zp2tf(z,p1,k1);
[h1,w1] = freqs(num1,den1);
figure; semilogx(w1,20*log10(abs(h1)));
title('Required filter');
xlabel('Frequency in rad/sec'); ylabel('Gain in dB');
```





Note that the cut-off frequency for the normalized filter is 1, whereas it is 0.5π = 1.57 for the required filter.

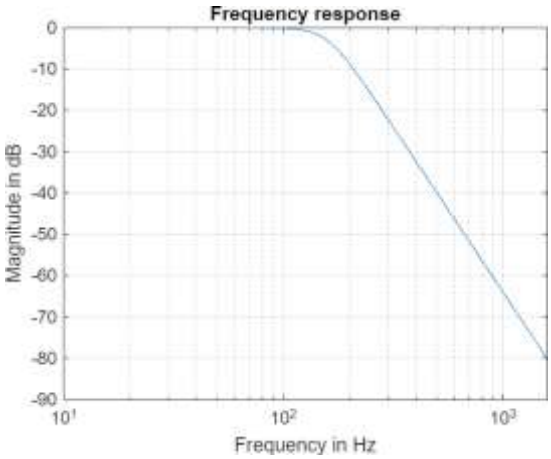
```
num1 =      0         0         0         0         0    9.5631
den1 =    1.0000    5.0832   12.9195   20.2939   19.7014    9.5631
```

The transfer function of the filter is:

$$H(s) = \frac{9.5631}{s^5 + 5.0832\,s^4 + 12.9195\,s^3 + 20.2939\,s^2 + 19.7014\,s + 9.5631}$$

Example-2: Design an analog Butterworth low-pass filter, given the following specifications and plot the frequency response. Passband edge = 100 Hz, Stopband edge = 500 Hz, Ripple in passband = 1 dB, Attenuation in the stopband is 40 dB.

```
clc; clear; close all;
fp = 100; wp = 2*pi*fp; % frequency in radians/sec
fs = 500; ws = 2*pi*fs;
rp = 1; rs = 40;
[N wc] = buttord(wp,ws,rp,rs,'s')          % obtain order and cut-off frequency
[b a] = butter(N,wc,'s');                  % obtain num and den coefficients
[h w] = freqs(b,a);                        % obtain frequency response in s-domain
f = w/(2*pi);                              % convert frequency to Hz
semilogx(f,20*log10(abs(h)));
grid on; title('Frequency response');
ylabel('Magnitude in dB');
xlabel('Frequency in Hz');
```



Results:

N = 4
wc/(2*pi) = 158.1159

Verify that the gain is -3 dB at f = 158 Hz, rp = -1 dB at 100 Hz and rs = -40 dB at 500 Hz

5. Exercises

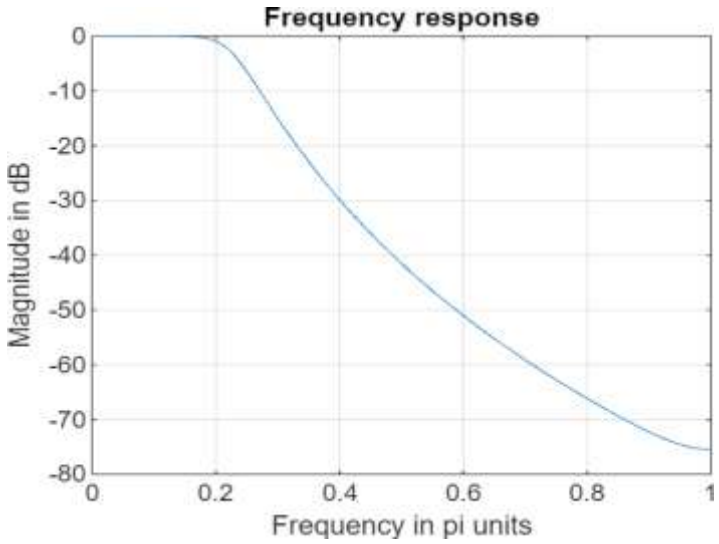
1. Given the following filter specifications: Passband edge = 200 Hz; Rp = 1dB; Stopband edge = 700Hz; Rs= 50 dB.
   a.) Design an analog lowpass Butterworth filter
   b.) Design an analog lowpass Chebyshev Type 1 filter
   c.) Design an analog lowpass Chebyshev Type 2 filter
   d.) Design an analog lowpass Elliptic filter
   Plot the frequency response for each case and compare them.

2. Given the following filter specifications: Passband edge: 1 kHz; Rp = 1dB; Stopband edge: 500Hz; Rs= 50 dB.
   a.) Design an analog high pass Butterworth filter
   b.) Design an analog high pass Chebyshev Type 1 filter

PART - B: Digital IIR Filters

Example-1: Design a low-pass digital filter using an analog Butterworth filter to satisfy ωp = 0.2π, ωs = 0.3π, Rp = 1 dB, Rs = 15 dB. Use the impulse invariance technique to digitize.

```
clc; clear; close all;
wp = 0.2*pi; ws =
0.3*pi; rp = 1; rs =
15;
[N wc] = buttord(wp,ws,rp,rs,'s')   % Order and cutoff frequency of analog
filter
[bs as] = butter(N,wc,'s');         % num and den coefficients in s-domain
[bz az] = impinvar(bs,as);          % coefficients in z-domain using impulse
invar (T=1)
[h w] = freqz(bz,az);               % frequency response in z-domain
plot((w/pi),20*log10(abs(h))); grid on;
title('Frequency response'); ylabel('Magnitude in dB'); xlabel('Frequency
in pi units');
```

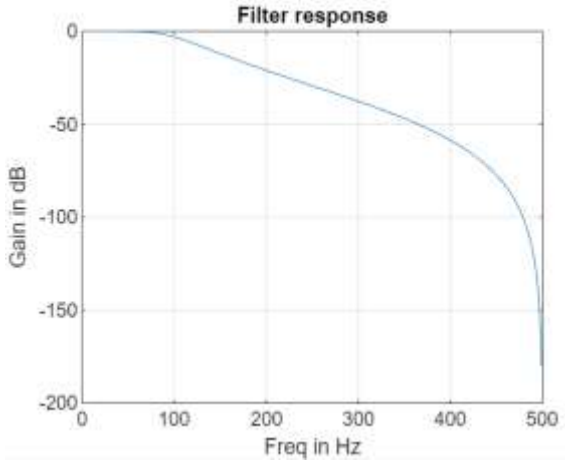


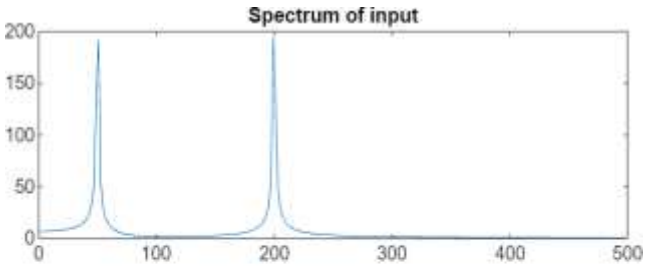
Zoom-in the graph and verify that gain is -1 dB at 0.2π (passband edge), -15 dB at 0.3π (stopband edge) and -3 dB at 0.225π (cutoff frequency)

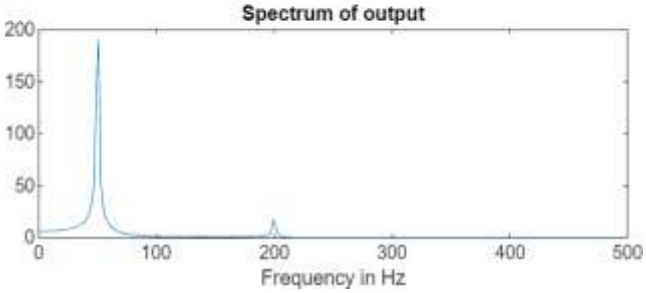
Example-2: Simulate a multitone signal with 50Hz and 200 Hz frequencies. Filter the signal through a 3rd order digital Butterworth LPF with 3-dB cutoff frequency 100 Hz. Plot the spectrum of the input and the filtered signal.

```
clc; clear; close all;
Fs = 1000;                          % sampling frequency
wc = 100/(Fs/2);                    % normalized cutoff frequency
[b a] = butter(3,wc);               % digital filter coefficients
using BLT
[h w] = freqz(b,a);                 % frequency response
```

```
f = (w/pi)*(Fs/2);                  % convert normalized frequency
into Hz
plot(f,20*log10(abs(h))); grid on;
ylabel('Gain in dB'); xlabel('Freq in Hz');
title('Filter response'); t = 0:(1/Fs):1; % time sequence
x = sin(2*pi*50*t) + sin(2*pi*200*t);     % input signal with 50Hz and
200Hz freqs
y = filter(b,a,x);                  % obtain the output of the filter
X = fft(x,512);                     % spectrum of the input
Y = fft(y,512);                     % spectrum of the output
L = length(X)/2;                    % take half of the samples
(0 to pi radians)
f = (0:(L-1))/L*(Fs/2);             % convert sample numbers
into frequency in Hz
figure; subplot(211); plot(f,abs(X(1:L)));
title('Spectrum of input');
subplot(212); plot(f,abs(Y(1:L)));
title('Spectrum of output'); xlabel('Frequency in Hz');
```

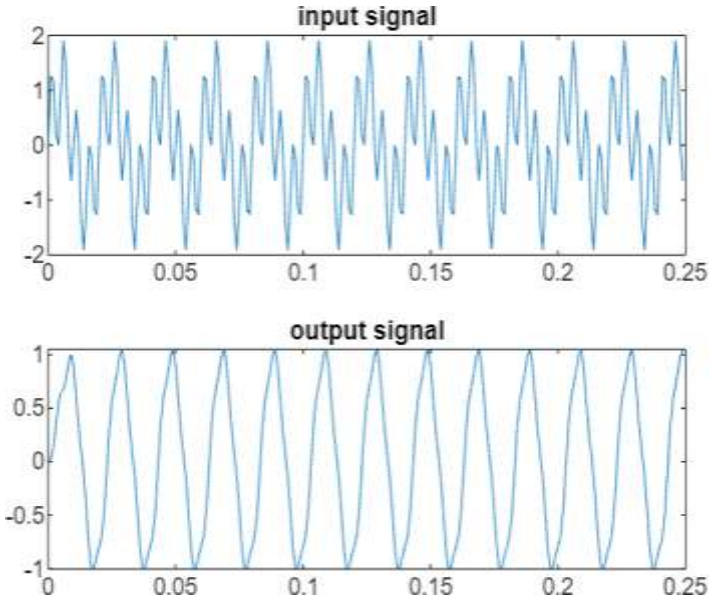






Note that input signal contains two frequencies 50 Hz and 200 Hz, whereas the output signal contains only one frequency 50 Hz. The other frequency has been filtered out by the LPF.

```
L=length(x)/4;              % add these 3 lines to the program to
plot the input & output
figure;
subplot(211);
plot(t(1:L),x(1:L));
title('input signal')
subplot(212);
plot(t(1:L),y(1:L));
title('output signal')
```



Exercises:

1.

$$H(s) = \frac{s + 1}{s^2 + 5\,s + 1}$$

Transform H(s) into H(z) using the impulse invariance technique in which T = 0.1s. First find the solution manually and then verify your result with the help of a MATLAB program. Also plot the magnitude responses of the analog and the resulting digital filter and comment on the result.

2. Design low-pass digital filter to satisfy: ωp = 0.3π, ωs = 0.5π, Rp = 1 dB, Rs = 20 dB, using:

   i) Chebyshev-I filter

   ii) Chebyshev-II filter

   iii) Elliptic filter.

   Use the impulse invariance technique with T = 1 for all above.

Experiment Execution [3]	Observation & Data Recording [3]	Results, Discussion and interpretation [2]	Exercise [2]	Total [10]

Experiment 5: FIR Filter Design

1. **Aim:** To design and study finite-duration impulse response (FIR) filters.

2. **Software used:** MATLAB

3. **Theory:**

A Finite Impulse Response (FIR) filter is a non-recursive digital filter whose output depends only on the present and past input samples. Since there is no feedback from the output to the input, the impulse response of an FIR filter has a finite duration, becoming zero after a finite number of samples. FIR filters are inherently stable and can be designed to exhibit an exactly linear phase response, making them suitable for applications where phase distortion must be minimized.

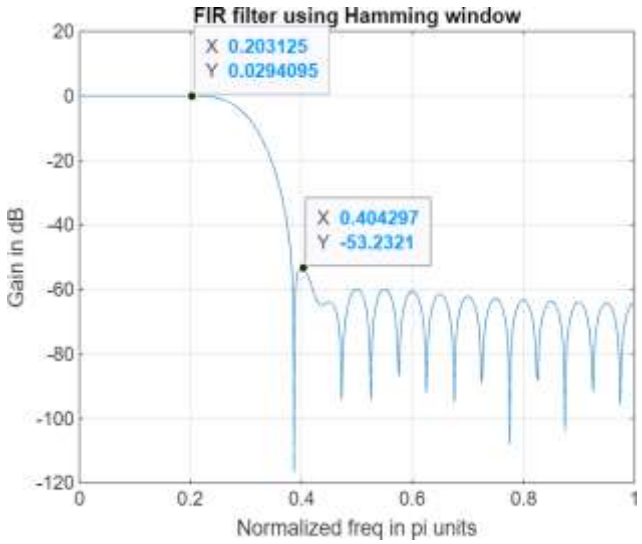
FIR filters are commonly designed using techniques such as the window method, frequency sampling method, and optimal approximation methods. In the window method, the ideal impulse response is truncated using window functions such as Rectangular, Hamming, Hanning, Blackman, or Kaiser windows to obtain the desired filter characteristics. The choice of window influences the filter's transition width and stopband attenuation.

Since FIR filters contain only zeros and no finite poles, they are always stable. Owing to their stability, linear phase property, and ease of implementation, FIR filters are widely used in digital communication systems, audio and speech processing, image processing, biomedical signal processing, and multirate signal processing applications.

4. **Program:**

Example-1: Design a low-pass digital FIR filter using Hamming window, whose desired specifications are: ωp = 0.2π rad, ωs = 0.4π rad, maximum passband ripple = 0.25 dB, minimum stopband attenuation = 50 dB.

```
clc; clear; close all;
wp = 0.2; ws = 0.4;              % normalized frequencies (in pi units)
N = ceil(8/(ws-wp));             % estimate the filter order
win = hamming(N+1);              % Hamming window used to get 53 dB
attenuation
wc = (wp+ws)/2;                  % wc taken as average of wp and ws
b = fir1(N,wc,"low",win);        % obtain numerator coefficients
[h w] = freqz(b,1);              % frequency response in z-domain.
Note: a = 1
plot((w/pi),20*log10(abs(h))); grid on;
title('FIR filter using Hamming window');
ylabel('Gain in dB'); xlabel('Normalized freq in pi units');
```

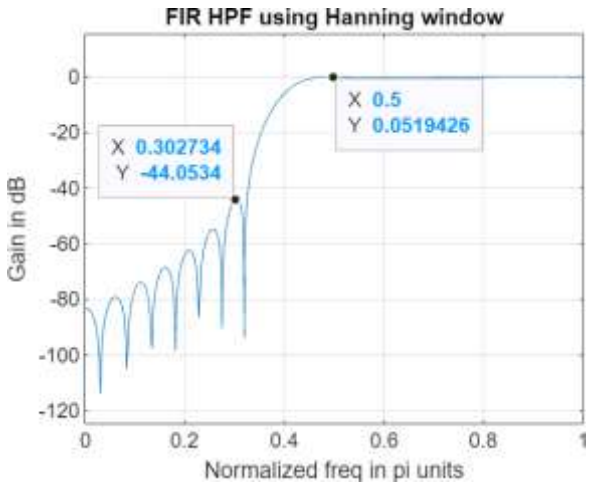


Note that the order of the filter is N = 40. Zoom-in the graph and find out the gains at ω = 0.2π is lesser than 0.25 dB, and the gain at ω = 0.4π is more than 50 dB, as per the required specifications.

Window	Stopband attenuation
Rectangular	21
Bartlett	25
Hanning	44
Hamming	53
Blackman	74

Example-2: Design a high-pass digital FIR filter using window method, whose desired specifications are: ωp = 0.5π rad, ωs = 0.3π rad, maximum passband ripple = 0.25 dB, minimum stopband attenuation = 40 dB.

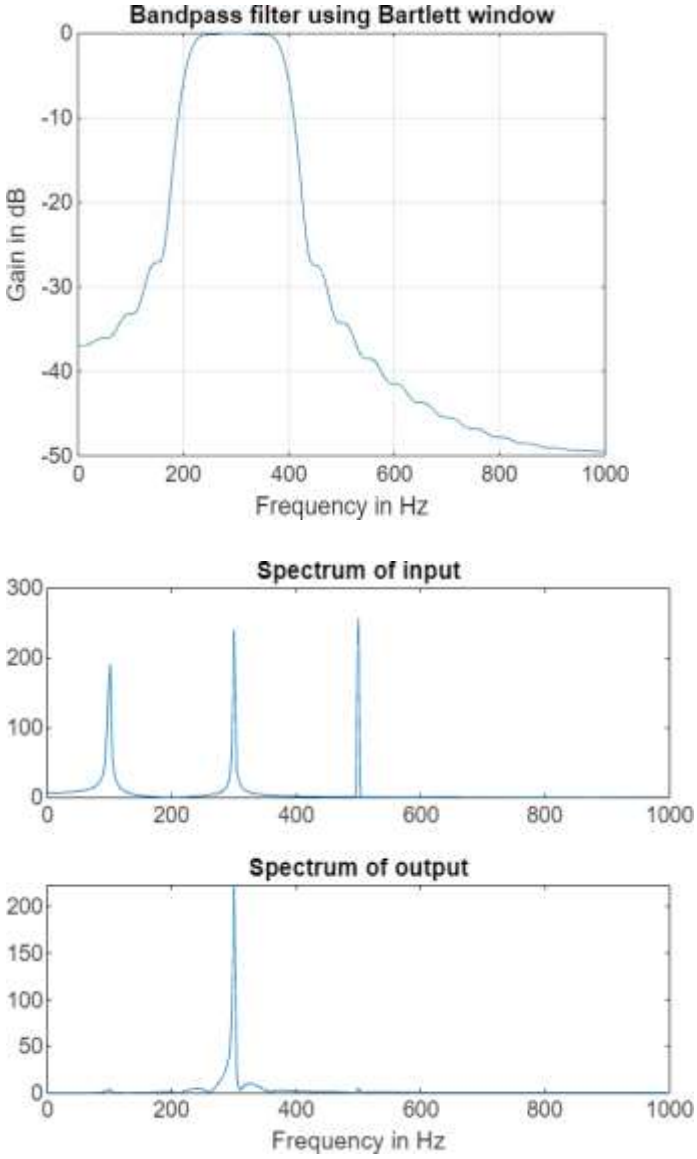
```
clc; clear; close all;
wp = 0.5; ws = 0.3;              % normalized frequencies (in pi units)
N = ceil(8/(wp-ws));             % estimate the order of the filter
win = hanning(N+1);              % Hanning window used to get 44 dB
attenuation
wc = (wp+ws)/2;                  % wc taken as average of wp and ws
b = fir1(N,wc,"high",win);       % obtain numerator coefficients
[h w] = freqz(b,1);              % frequency response in z-domain
plot((w/pi),20*log10(abs(h)));
grid on;
title('FIR HPF using Hanning window');
ylabel('Gain in dB'); xlabel('Normalized freq in pi units');
```



Example-3: Certain signal has 100 Hz, 300 Hz and 500 Hz components. Design a suitable FIR filter to select only the 300 Hz component. Filter the signal using the filter and plot the spectrum of the input and the filtered signal. Use sampling frequency 2000 Hz.

```
clc; clear; close all;
Fs = 2000; f1 = 100; f2 = 300; f3 =
500; t = 0:(1/Fs):1;
x = sin(2*pi*f1*t)+sin(2*pi*f2*t)+sin(2*pi*f3*t); % input signal
fp = [250 350]; fs = [150 450];      % band edge frequencies
wp = fp/(Fs/2); ws = fs/(Fs/2);      % normalize the frequencies
wc = (wp+ws)/2;                      % cutoff frequency
tw = min(abs(wp-ws));                % transition width
N = ceil(8/tw);                      % order of the filter
win = bartlett(N+1);    % Bartlett window used to get 25 dB attenuation
b = fir1(N,wc,"bandpass",win);      % filter coefficients
[h w] = freqz(b,1,512,Fs);          % 512-point frequency response,
w in Hz

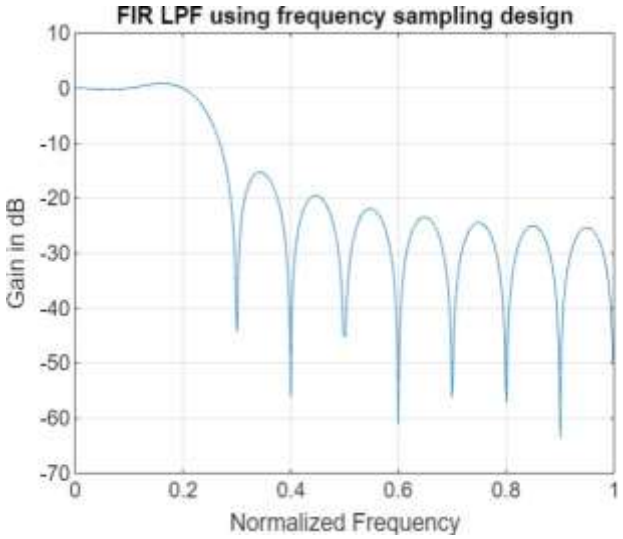
plot(w,20*log10(abs(h))); grid on;
title('Bandpass filter using Bartlett window');
ylabel('Gain in dB'); xlabel('Frequency in Hz');
y = filter(b,1,x);                   % filtered output
X = fft(x,512); Y = fft(y,512);
L = length(X)/2; f = (0:(L-1))/L*(Fs/2);
figure; subplot(211); plot(f,abs(X(1:L)));
title('Spectrum of input');
subplot(212); plot(f,abs(Y(1:L)));
title('Spectrum of output'); xlabel('Frequency in Hz');
```



Example-4: Design the filter specified in problem no.1 using frequency sampling design technique assuming linear phase. Desired specifications are: ωp = 0.2π rad, ωs = 0.4π rad.

```
clc, clear all, close all;
M = 20;                                  % assume
Hr = [1 1 1 zeros(1,15) 1 1];            % desired frequency response
magnitude
k1 = 0:floor((M-1)/2);
k2 = floor((M-1)/2)+1:(M-1);
a = (M-1)/2;
b = [-a*2*pi*k1/M, a*2*pi*(M-k2)/M]; % desired frequency response
```

```
phase angles
H = Hr.*exp(j*b);            % multiply magnitude and phase to get H
b = real(ifft(H,M));         % Inverse DFT to get b coefficients
[h w]=freqz(b,1);                    % frequency response plot
plot(w/pi,20*log10(abs(h)));
grid on; title('FIR LPF using frequency sampling
design'); xlabel('Normalized Frequency'),
ylabel('Gain in dB');
```



5. Exercises

1. Design and plot the frequency response of a low-pass digital FIR filter whose desired specifications are: $\omega p = 0.2\pi$, $\omega s = 0.4\pi$, using (a) Rectangular window, (b) Hanning window, (c) Kaiser window. Compare the results.

2. Design and plot the frequency response of a linear phase band-pass filter whose specifications are as follows: Passband edges: [400Hz 500Hz], Stopband edges: [300Hz 600Hz], Rp = 0.5dB and As = 40dB. The sampling frequency is 2 kHz, using (a) Hamming window (b) Kaiser window.

3. In example problem 4, the attenuation in the stopband is found to be inadequate. To improve it, a few non-zero points are considered in the transition band. Use Hr[k] = {1, 1, 1, 0.5, 0.1, zeros(1,11), 0.1, 0.5, 1, 1} and design the filter using frequency sampling technique. Compare the results and find out whether there is any improvement. If there is, then at what cost?

4. Design the filter specified in exercise problem 2 using frequency sampling technique.

Experiment Execution [3]	Observation & Data Recording [3]	Results, Discussion and interpretation [2]	Exercise [2]	Total [10]

Experiment 6: Image Processing

1. **Aim:** To study the analysis of signals and systems in Z-domain

2. **Software used:** MATLAB

3. **Theory:**

Digital image processing involves manipulation and analysis of digital images to improve their quality or extract useful information. A digital image is represented as a two-dimensional array of pixels, where each pixel corresponds to an intensity or color value. Image processing techniques are widely used in applications such as medical imaging, remote sensing, surveillance, industrial inspection, computer vision, and multimedia systems.

Basic image processing operations include image acquisition, grayscale conversion, histogram analysis, intensity transformations, filtering, and geometric transformations. Histogram analysis provides information about the distribution of pixel intensities and is useful for evaluating image contrast. Histogram equalization enhances image contrast by redistributing intensity levels, while negative, logarithmic, and gamma transformations modify image brightness and contrast for improved visualization. Spatial filtering techniques, such as mean, Gaussian, and median filtering, are used to smooth images, reduce noise, or preserve important image features. Other operations, including cropping, resizing, and color component extraction, enable the manipulation and analysis of specific image regions or color information. These fundamental image processing techniques provide the basis for advanced applications such as object recognition, image segmentation, compression, and computer vision.

4. **Example:**

    1. **Image Reading, Display, and Basic Info Extraction**

```
img = imread('peppers.png');
imshow(img);
title('Original Image');

% Get image information
disp('Image size:');
disp(size(img));
disp('Image class:');
disp(class(img));
```

    2. **Convert to Grayscale and Histogram Analysis**

```
img = imread('peppers.png');
gray_img = rgb2gray(img);
imshow(gray_img);
title('Grayscale Image');
```

```
figure;
imhist(gray_img);
title('Histogram of Grayscale Image');
```

3. Image Enhancement using Histogram Equalization

```
img = imread('peppers.png');
gray_img = rgb2gray(img);
eq_img = histeq(gray_img);
% Display both images
subplot(1,2,1);
imshow(gray_img);
title('Original Grayscale');

subplot(1,2,2);
imshow(eq_img);
title('Equalized Image');
% Plot histograms
figure;
subplot(1,2,1);
imhist(gray_img);
title('Original Histogram');

subplot(1,2,2);
imhist(eq_img);
title('Equalized Histogram');
```

4. Image Filtering using Convolution (Smoothing and Sharpening)

```
% Smoothing filter
img = imread('peppers.png');
gray_img = rgb2gray(img);
h_smooth = fspecial('average', [5 5]);
smooth_img = imfilter(gray_img, h_smooth);

% Sharpening filter
h_sharp = fspecial('unsharp');
sharp_img = imfilter(gray_img, h_sharp);

% Display results
subplot(1,3,1);
imshow(gray_img);
title('Original');

subplot(1,3,2);
imshow(smooth_img);
title('Smoothed');
subplot(1,3,3);
```

```
imshow(sharp_img);
title('Sharpened');
```

5. Image Negative Transformation

```
% Read and convert to grayscale
img = rgb2gray(imread('peppers.png'));

% Create negative image
negative_img = 255 - img;

% Display original and negative
subplot(1,2,1);
imshow(img);
title('Original Image');

subplot(1,2,2);
imshow(negative_img);
title('Negative Image');
```

6. Logarithmic and Power-Law (Gamma) Transformations

```
% Read and convert to grayscale
img = im2double(rgb2gray(imread('peppers.png')));

% Log transformation
log_img = log(1 + img);
log_img = log_img / max(log_img(:));  % Normalize

% Gamma correction
gamma = 0.5;  % Try values like 0.4, 1.0, 2.0
gamma_img = img .^ gamma;

% Display results
figure;
subplot(1,3,1); imshow(img); title('Original');
subplot(1,3,2); imshow(log_img); title('Log Transform');
subplot(1,3,3); imshow(gamma_img); title(['Gamma Transform (\gamma = ',
num2str(gamma), ')']);
```

7. Image Blurring using Gaussian Filter

```
% Read and convert to grayscale
img = rgb2gray(imread('peppers.png'));

% Create Gaussian filter
h = fspecial('gaussian', [5 5], 1.0);  % size, sigma

% Apply filter
```

```
blurred = imfilter(img, h);

% Display results
figure;
subplot(1,2,1); imshow(img); title('Original');
subplot(1,2,2); imshow(blurred); title('Gaussian Blurred');
```

8. Image Cropping and Region of Interest (ROI) Selection

```
img = imread('peppers.png');

% Select ROI manually
figure;
imshow(img);
title('Select ROI and Double Click');
roi = imcrop;  % Manually select ROI

% Display cropped ROI
figure;
imshow(roi);
title('Cropped ROI');
```

9. Median Filtering for Noise Removal

```
% Read and convert to grayscale
img = rgb2gray(imread('peppers.png'));

% Add salt-and-pepper noise
noisy_img = imnoise(img, 'salt & pepper', 0.05);

% Apply median filter
filtered_img = medfilt2(noisy_img, [3 3]);

% Display
figure;
subplot(1,3,1); imshow(img); title('Original');
subplot(1,3,2); imshow(noisy_img); title('Noisy Image');
subplot(1,3,3); imshow(filtered_img); title('After Median Filtering');
```

10. Extract RGB from a colour image

```
img = imread('peppers.png');

r = img(:,:,1);
g = img(:,:,2);
b = img(:,:,3);

% Display
figure;
```

```
subplot(2,2,1); imshow(img); title('Original');
subplot(2,2,2); imshow(r); title('Red colour extraction');
subplot(2,2,3); imshow(g); title('Green colour extraction');
subplot(2,2,4); imshow(b); title('Blue colour extraction');
```

5. Exercises

1. Image Brightness Adjustment - Read a color image and convert it to grayscale. Increase the brightness by adding a constant value (e.g., 40) to each pixel. Decrease the brightness by subtracting the same constant. Display the original, brighter, and darker images.

2. Image Rotation and Flipping - Perform basic geometric transformations on an image like rotating the image by 90° and 180° and flipping the image horizontally and vertically. Display all transformed images along with the original.

3. Comparison of Mean and Median Filtering - Add salt-and-pepper noise to a grayscale image. Apply a mean (average) filter and a median filter separately. Display the original, noisy, mean-filtered, and median-filtered images. Compare the quality of the filtered images.

4. Image Resizing Using Interpolation - Resize the image to half of its original size and then twice its original size. Display the original, reduced, and enlarged images.

Experiment Execution [3]	Observation & Data Recording [3]	Results, Discussion and interpretation [2]	Exercise [2]	Total [10]

Introduction to Code Composer Studio (CCS)

1. Aim:

1. To get familiarized with code composer studio.
2. To implement basic DSP programs and verify the simulation.

2. Introduction

Code composer is the DSP industry's first fully integrated development environment (IDE) with DSP-specific functionality. With a familiar environment liked MSbased C++TM, Code Composer lets you edit, build, debug, profile and manage project from a single unified environment. Other unique features include graphical signal analysis, injection/extraction of data signals via I/O, multi-processor debugging, automated testing and customization via a C-interpretive scripting language and much more.

Code Composer Studio (CCS) Features:
1. IDE
2. Debug IDE
3. Advanced watch windows
4. Integrated editor
5. File I/O, Probe points, and graphical algorithm scope probes.
6. Advanced graphical signal analysis
7. Interactive profiling
8. Automated testing and customization via scripting
9. Visual project management system
10. Compile in the background while editing and debugging
11. Multi-processor debugging
12. Help on the target DSP

Documents for Reference:
Spru509-Code Composer Studio getting started guide.
Spru189 -TMS320C6000 CPU and Instruction set guide.
Spru189 -TMS320C6000 Peripherals guide.
Slws106d- codec (TLV320AIC23) data manual.
Spru402- Programmer's reference guide.
Sprs186j- TMS320C6713 DSP
DSP Tools: www.ti.com/c6000
Software: http://processors.wiki.ti.com/index.php/Category:Code_Composer_Studio_v6 or higher.
Processor: http://www.ti.com/product/omap-1138

Example:
Write a C code to generate a sinusoidal signal and implement the same on TMS320C6748 DSP kit.
• Procedure to create a simple non-real time project - Sine wave Generation:
• Once you run the program, 128 no's of sine samples has been printed in the Console Window.
• To view the values in graph, Go to Tools - Graph - Single Time.
• Set the Graph Properties as shown in the Picture.
• Buffer size: 128 (No. of samples)
• DSP Data Type: 32-Bit floating point
• Start Address: m
• Display Data Size: 128
• Click: Ok
• Graph will be plotted.

Exercise:
Write a C code to generate a random wave and implement the same on TMS320C6748 DSP kit.

Experiment 7: DFT computation using TMS320C6748 DSP starter kit

1. **Aim:** To compute n-point DFT of a given sequence using TMS320C6748 DSP starter kit.

2. **Hardware/ Software used:** TMS320C6748 DSP starter kit, PC with code composer studio (CCS version 3.1)

3. **Theory:**

DFT of a discrete time signal transforms the signal in time domain into frequency domain signal.

DFT EQUATION:

The sequence of N complex numbers x_0, ..., x_{N-1} is transformed into another sequence of N complex numbers according to the DFT formula:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j\frac{2\pi}{N}kn}, \qquad k = 0, 1, \ldots, N-1$$

where:

- $x[n]$: Input discrete-time sequence
- $X[k]$: DFT of the sequence
- N : Number of samples
- n : Time-domain sample index
- k : Frequency-domain sample index
- $j = \sqrt{-1}$

The inverse discrete Fourier transform (IDFT) is given by:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]\, e^{j\frac{2\pi}{N}kn}, \qquad n = 0, 1, \ldots, N-1$$

4. **Example:** Compute n-point DFT of the sequence (1 1 2 -2 3 0 5 1) and the code needs to be implemented on the DSP processor.

5. **Exercise:** Compute n-point IDFT of the sequence (1 1 2 -2 3 0 5 1) and the code needs to be implemented on the DSP processor.

Experiment Execution [3]	Observation & Data Recording [3]	Results, Discussion and interpretation [2]	Exercise [2]	Total [10]

Experiment 8: IIR Filter Implementation using TMS320C6748 DSP kit

1. **Aim:** To design and implement a Digital IIR Filter using the DSP kit and observe its frequency response.

2. **Hardware/ Software used:** TMS320C6748 DSP starter kit, PC with code composer studio (CCS version 3.1), CRO and function generator.

3. **Theory:**

   Infinite Impulse response (IIR) filter possesses an output response to an impulse which is of an infinite duration. The impulse response is "infinite" since there is feedback in the filter, which is if you put in an impulse, then its output must produce for infinite duration of time. The aim of this laboratory exercise is to design and implement a Digital IIR Filter & observe its frequency response. In this experiment we design a simple IIR filter to stop or attenuate required band of frequencies components and pass the frequency components which are outside the required band.

4. **Example 1: To design and implement IIR filter on DSK6748 board.**

Algorithm to implement:

Realize the Butter worth band pass IIR filter by implementing the difference equation $y[n] = b_0x[n] + b_1x[n-1]+b_2x[n-2]-a_1y[n-1]-a_2y[n-2]$ where $b_0 – b_2$, a_0-a_2 are feed forward and feedback word coefficients respectively [Assume 2nd order of filter].These coefficients are calculated using MATLAB. A direct form I implementation approach is taken.

Step 1 - Initialize the McASP, the DSP board and the on board codec.

Step 2 - Initialize the discrete time system, that is, specify the initial conditions. Generally zero initial conditions are assumed.

Step 3 - Take sampled data from codec while input is fed to DSP kit from the signal generator. Since Codec is stereo, take average of input data read from left and right channel. Store sampled data at a memory location.

Step 4 - Perform filter operation using above said difference equation and store filter Output at a memory location.

Step 5 - Output the value to codec (left channel and right channel) and view the output at Oscilloscope.

Step 6 - Go to step 3.

Procedure:

- Add bs1800int.cof file that contains filter co-efficient (will be in IIR filter folder)
- After successful generation of .out file connect target to host PC
- Connect signal generator to LINE INPUT with 2Vp-p Sine Wave frequency is set according to filter co-efficient.
- Connect CRO to LINE OUT with volt/div at 1 and time/div at 5msec.

Note:

To convert the floating-point coefficients (obtained from MATLAB program) into its equivalent 16-bit fixed point representation, multiply coefficients by a factor = 2^{15}=32768.

5. Exercise:

1. Design an IIR Butterworth high-pass filter having cut-off frequency of 7000Hz. Here the IIR HPF must be written using MATLAB and the coefficients must be first converted to its fixed-point equivalent and then copied.

2. Write C-program to generate four (choose any four frequency components below 1000Hz) different tones at the speaker out, which are controlled by the onboard DIP switch (SW3).

Experiment Execution [3]	Observation & Data Recording [3]	Results, Discussion and interpretation [2]	Exercise [2]	Total [10]

Experiment 9: FIR Filter Implementation using TMS320C6748 DSK

1. **Aim:** To implement FIR filters on the DSK and verify their working.

2. **Hardware/ Software used:** TMS320C6748 DSP starter kit, PC with code composer studio (CCS version 3.1), CRO and function generator.

3. **Theory:**

The FIR filters are of non-recursive type, whereby the present output sample is depending on the present input sample and previous input samples. The transfer function of a FIR causal filter is given by,

$$H(Z) = \sum_{n=0}^{N-1} h(n)_{Z^{-n}}$$

Where h(n) is the impulse response of the filter.

The Fourier transform of h(n) is

$$H(e^{jw}) = \sum_{n=0}^{N-1} h(n)e^{-j\omega n}$$

In the design of FIR filters most used approach is using windows. The desired frequency response $H_d(e^{jw})$ of a filter is periodic in frequency and can be expanded in Fourier series. The resultant series is given by,

$$h_d(n) = (1/2\pi) \int_{-\pi}^{\pi} H(ejw)e^{jwn}dw$$

4. **Example**

To design and implement FIR filter on DSK6748 board.

Designing an FIR filter:

Following are the steps to design linear phase FIR filters Using Windowing Method

1. Clearly specify the filter specifications.
   Eg: Order = 30; Sampling Rate = 8000 samples/sec Cut off Freq. = 400 Hz.
2. Compute the cut-off frequency W_c

   Eg: Wc = 2*pi* fc / Fs

   = 2*pi* 400/8000 = 0.1*pi
3. Compute the desired Impulse Response h d (n) using particular Window
   Eg: b_rect1=fir1(order, Wc , 'high',boxcar(31));
4. Convolve input sequence with truncated Impulse Response x (n)*h (n).

Using MATLAB to determine filter coefficients:

Using FIR1 Function on MATLAB.

- B = FIR1(N,Wn) designs an N'th order lowpass FIR digital filter and returns the filter coefficients in length N+1 vector B.
- The cut-off frequency Wn must be between 0 < Wn < 1.0, with 1.0 corresponding to half the sample rate. The filter B is real and has linear phase, i.e., even symmetric coefficients obeying B(k) = B(N+2-k), k = 1,2,...,N+1.
- If Wn is a two-element vector, Wn = [W1 W2], FIR1 returns an order N bandpass filter with passband W1 < W < W2. B = FIR1(N,Wn,'high') designs a highpass filter. B = FIR1(N,Wn,'stop') is a bandstop filter if Wn = [W1 W2].
- If Wn is a multi-element vector, Wn = [W1 W2 W3 W4 W5 ... WN], FIR1 returns an order N multiband filter with bands 0 < W < W1, W1 < W < W2, ..., WN < W < 1. B = FIR1(N,Wn,'DC-1') makes the first band a passband. B = FIR1(N,Wn,'DC-0') makes the first band a stopband.
- For filters with a passband near Fs/2, e.g., highpass and bandstop filters, N must be even.
- By default FIR1 uses a Hamming window. Other available windows, including Boxcar, Hanning, Bartlett, Blackman, Kaiser and Chebwin can be specified with an optional trailing argument. For example, B = FIR1(N,Wn,kaiser(N+1,4)) uses a Kaiser window with beta=4. B = FIR1(N,Wn,'high',chebwin(N+1,R)) uses a Chebyshev window.
- By default, the filter is scaled so the center of the first pass band has magnitude exactly one after windowing. Use a trailing 'noscale' argument to prevent this scaling, e.g. B = FIR1(N,Wn,'noscale'), B = FIR1(N,Wn,'high','noscale'), B = FIR1(N,Wn,wind,'noscale').

Procedure:

- After successful generation of .out file connect target to host PC.
- Connect signal generator to LINE INPUT with 2Vp-p Sine Wave frequency is set according to filter co-efficient.
- Connect CRO to LINE OUT with volt/div at 1 and time/div at 5msec.

5. Exercise:

1. Implement an FIR low-pass filter having cut-off frequency of 1000Hz. This should contain the coefficients for FIR LPF which are obtained from MATLAB using fir1 function using rectangular window-based design.

2. Realization of an FIR filter (any type) to meet given specifications. The input can be a signal from function generator / speech signal.

Experiment Execution [3]	Observation & Data Recording [3]	Results, Discussion and interpretation [2]	Exercise [2]	Total [10]

Experiment 10: Audio Processing using TMS320C6748 DSK

1. **Aim:** To configure the codec AIC3106 for a talk through program using the board support library.

2. **Hardware/ Software used:** TMS320C6748 DSP starter kit, PC with code composer studio (CCS version 3.1), CRO, Audio Source, Speakers, headphone and Signal Generator.

3. **Theory:**

Speech is an acoustic signal, which is produced when air is forced from the human lungs through the vocal chords (glottis) and along the vocal tract. The human voice conveys information which is received by our ears and decoded by the brain according to the sounds used in the language used. Speech signal is a highly non-stationary signal, having many frequency components. It is required to find out which frequency is present at what time period. This is called time resolution of frequency. Speech signal is actually slowly time-varying signal, i.e., when observed over a sufficiently short duration of time, it is fairly stationary. Therefore, we take short-time Fourier transform (STFT) of speech signal to find its frequency content.

4. **Example:** To configure the codec AIC3106 for a talk through program using the board support library.

Procedure:

- All the Real time implementations covered in the Implementations module follow code Configuration using board support library.
- The board Support Library (CSL) is a collection of functions, macros, and symbols used to configure and control on-chip peripherals.
- The goal is peripheral ease of use, shortened development time, portability, hardware abstraction, and some level of standardization and compatibility among TI devices.
- Connect one end of a stereo cable to the LINE IN(J55) of the kit and other end to PC (or function generator).
- Connect the headphone (or CRO) to the LINE OUT(J56) of the kit.
- Connect the power supply to the board.
- Connect the External JTAG XDS100v2 with board and PC.

General Steps for Real Time Programs:

R1. Create project.

- Go to File ->new->CCS project and enter the details as done in NR3 but only change in “is Advanced Settings” choose the Linker command file: as “linker_dsp.cmd” (Browse from Supporting files folder given at the time of installation).

R2. Create source file to write code.

- Once the program is written, save it.

R3. Now add the following library files and supporting files of codec to your project. To add files goto Project-> Add files.

R4. Right click on the project name and choose “Show Build Settings..” -> Build-> C6000 Compiler-> Include Options -> Click “Add” as shown in the figure.

R5. Build Project, Load and Resume the project.

- After clicking the Resume option, play the music now. You can hear the song from the headphones without any noise. Alternatively, you can also connect function generator to the LINE IN and CRO to the LINE OUT. At this time, you can observe same wave appearing on the CRO which is given in function generator. Thus, the CODEC is verified. You can also try changing the sampling frequency, gain of the codec.
- Note: if you want to play loop signal only in right channel then change the 6th line to output_right_sample(sample). If you want to play loop signal only in left channel then change the 6th line to output_left_sample(sample).

5. Exercise:

1. Configure the codec AIC3106 for creating ECO effect through program using the board support library.

2. Configure the codec AIC3106 for creating Delay effect through program using the board support library.

Experiment Execution [3]	Observation & Data Recording [3]	Results, Discussion and interpretation [2]	Exercise [2]	Total [10]